Programmation GPGPU

Jonathan Caux

jcaux@adobe.com

Contexte du cours

Qui suis-je

- Jonathan Caux
- ZZ2 promo 2008
- Thèse au LIMOS soutenu en 2012
  - Cours C++
  - Cours GPGPU
- Depuis principalement dév' C++
  - Murex, progiciel de finance (dév' C++)
  - AVM Up, téléphonie (dév' C++)
  - Thales service, ESN du groupe Thales (tech lead C++)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Adobe 3D&I

- Senior dev’ à Clermont depuis 2021 chez Adobe
- Division 3D&I (3D and Immersive) issue du rachat d’Allegorithmic
  - Allegorithmic, entreprise de texturing 3D créée à Clermont et d’abord hébergée à l’ISIMA
- Gamme de produit autour du texturing 3D
  - Designer, Painter (Clermont)
  - Sampler (Lyon)
  - Pleins d’autres logiciels et outils, ici et ailleurs

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Adobe Substance 3D Designer

- Équipe (au sens large) de 11 personnes :
  - 1 tech lead + 5 dev’ (dont 2 seniors)
  - 2 PM (plus ou moins PO/scrum master)
  - 2 ingé’ qualité
  - 1 UX/UI designer

- Logiciel de création de texture 3D
  - C++ / Qt / Google Test
  - CMake / git / Jenkins

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Contenu du cours

- Cours réalisé en 2010 pour la première fois
  - Domaine novateur à l'époque « Bémol de la nouvelle technologie : quelle pérennité ? » (cours GPGPU, 2010)
  - Domaine bien maitrisé maintenant
- Explication du pourquoi du GPGPU + Programmation en CUDA
- Objectif double :
  - Savoir implémenter un algo sur GPU
  - Savoir quand et comment implémenter un algo sur GPU (force et faiblesse des GPU)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Contenu du cours

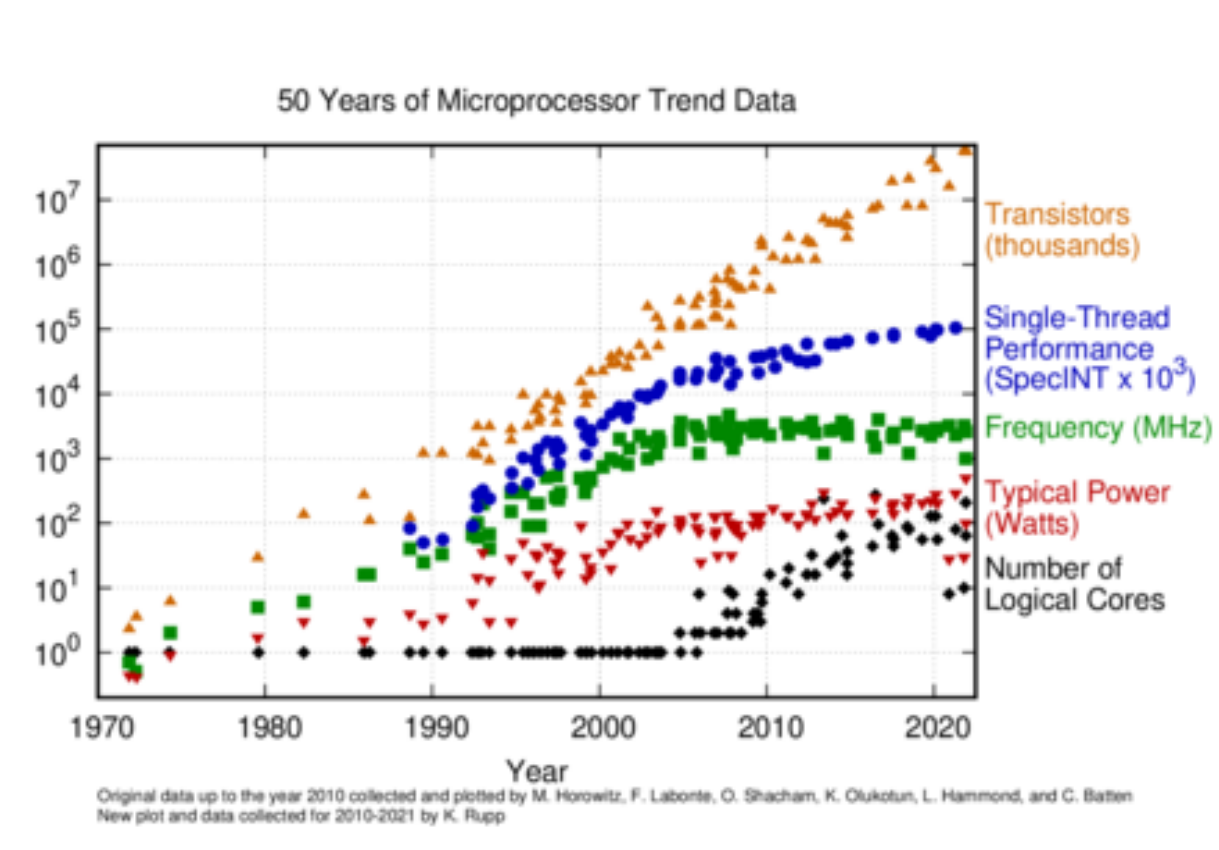
- N’hésitez pas à remonter vos remarques
- Loin de tout savoir, contenu très largement inspiré de la documentation CUDA
- Évaluation : examen écrit + projet ? Quid de l’IA ?

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Historique du contexte GPGPU

Evolution des microprocesseurs - courbes

- Augmentation exponentielle jusqu'en 2005
- A partir de 2005 :
  - Augmentation + lente des perf' single thread
  - Multiplication des cœurs

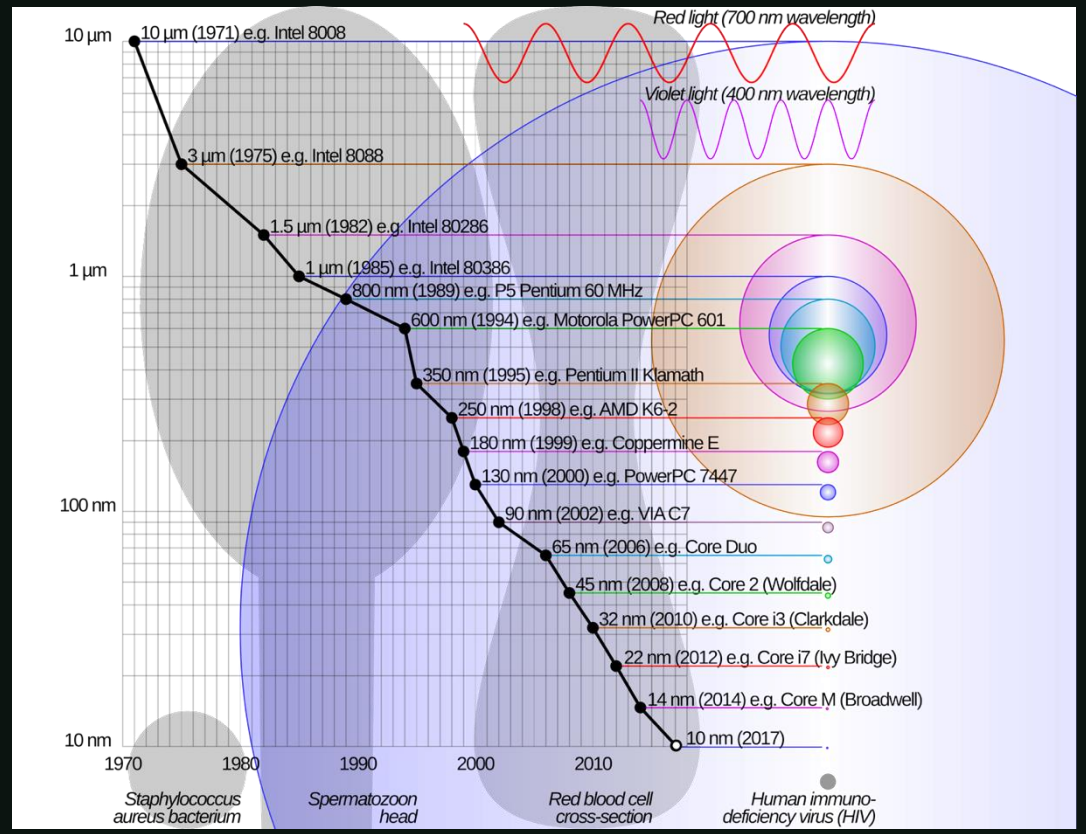


https://fr.wikibooks.org/wiki/Fonctionnement_d%27un_ordinateur/La_loi_de_Moore_et_les_tendances_technologiques

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Evolution des microprocesseurs - gravure

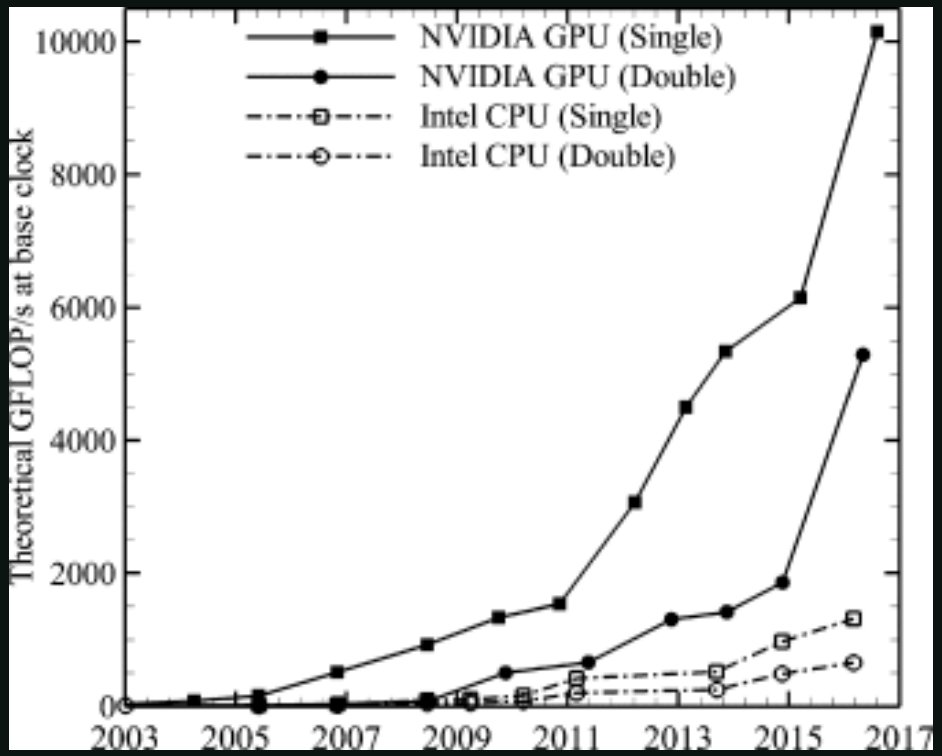
- Augmentation exponentielle jusqu'en 2005
- Gravure de plus en plus fine
  - Toujours plus de transistor au m2
  - (5nm sans doute la limite)
- Problématique d'alimentation de ces transistors toujours plus fins.



https://fr.wikibooks.org/wiki/Fonctionnement_d%27un_ordinateur/La_loi_de_Moore_et_les_tendances_technologiques

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Puissance de calcul CPU vs GPU

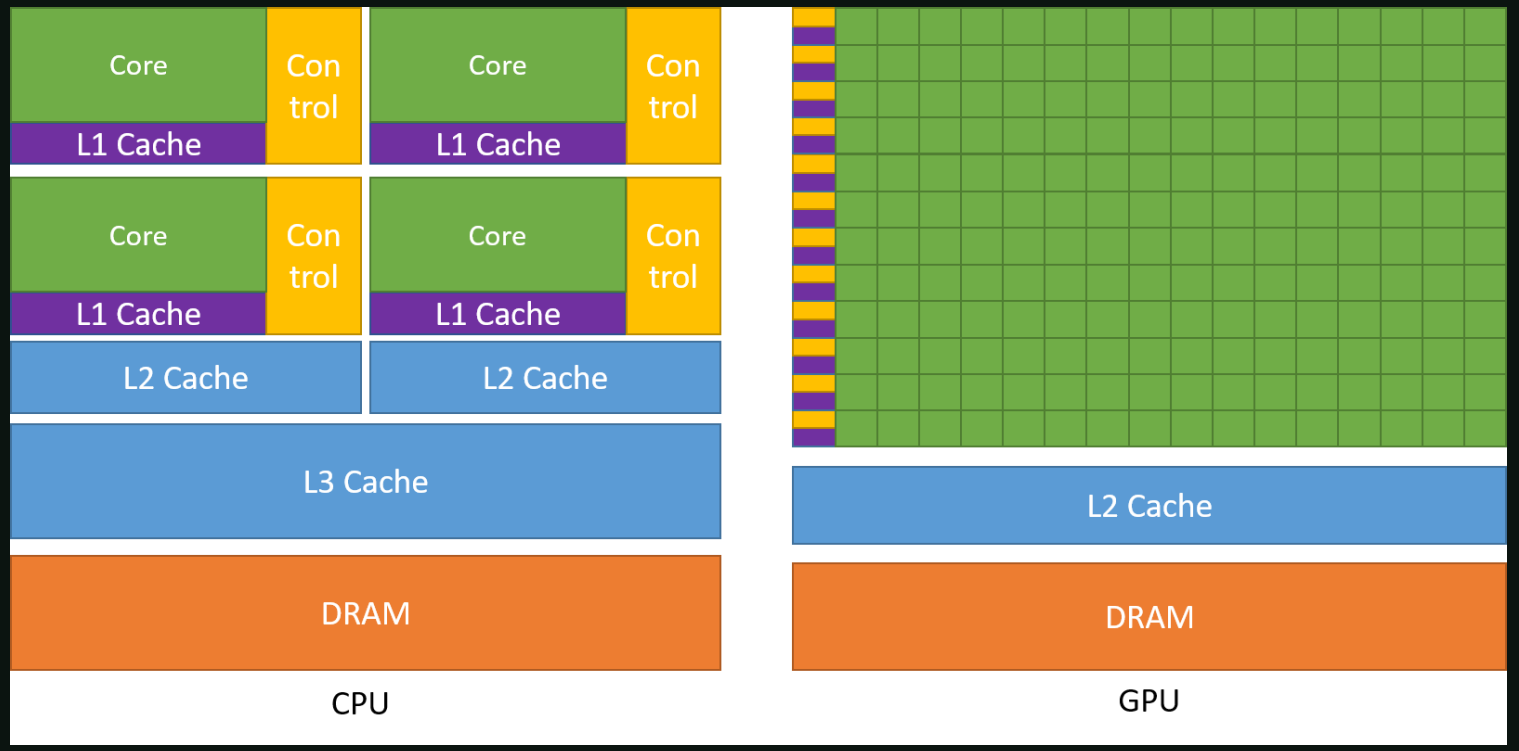


An investigation of hybrid CPU-GPU solvers for supersonic reacting flow simulation with detailed chemical kinetics

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Organisation CPU vs GPU - aperçu

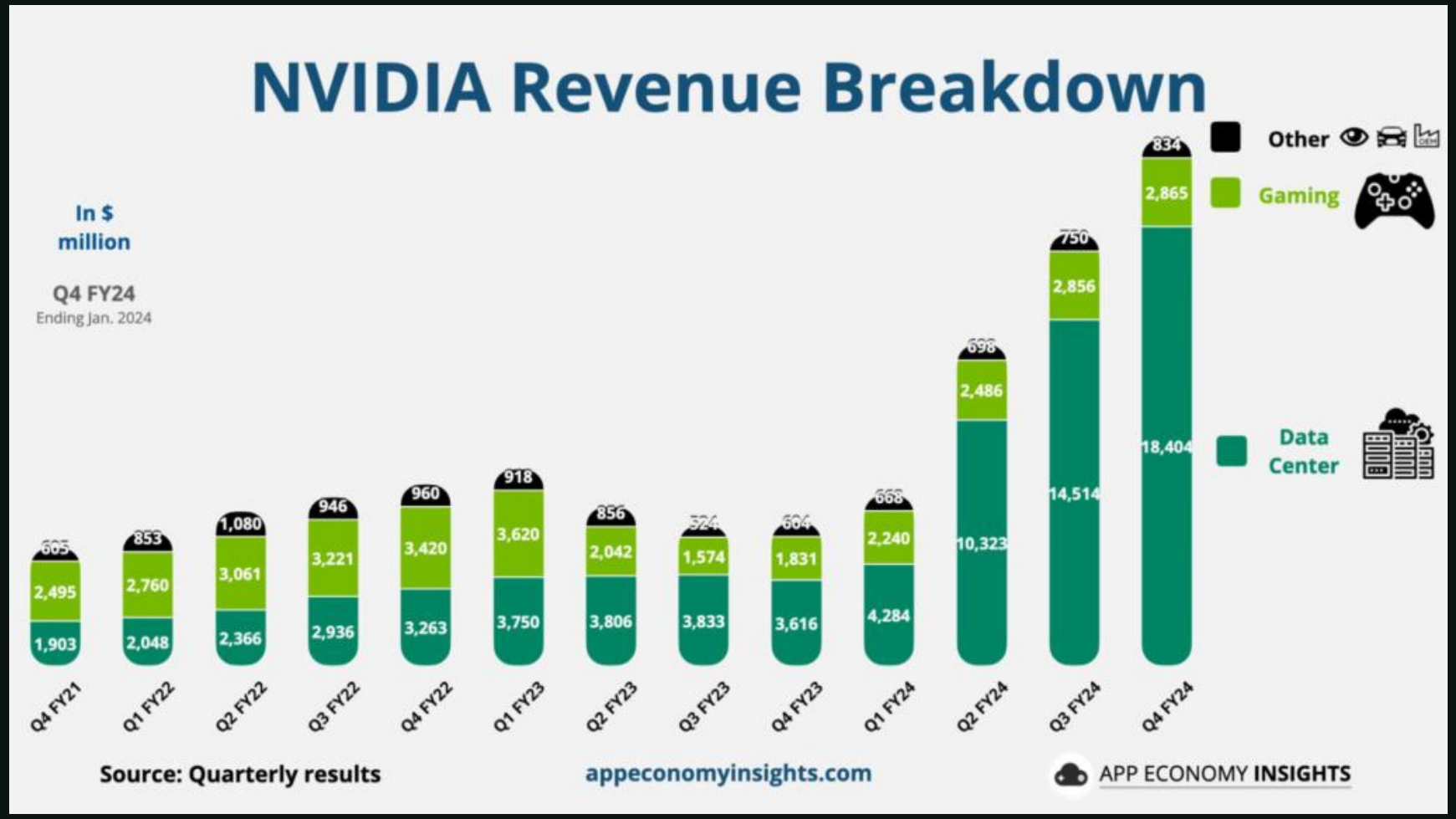
- CPU optimisé pour le contexte switching
- GPU optimisé pour le traitement d'un même jeu d'instruction sur des données différentes



https://docs.nvidia.com/cuda/cuda-c-programming-guide/

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

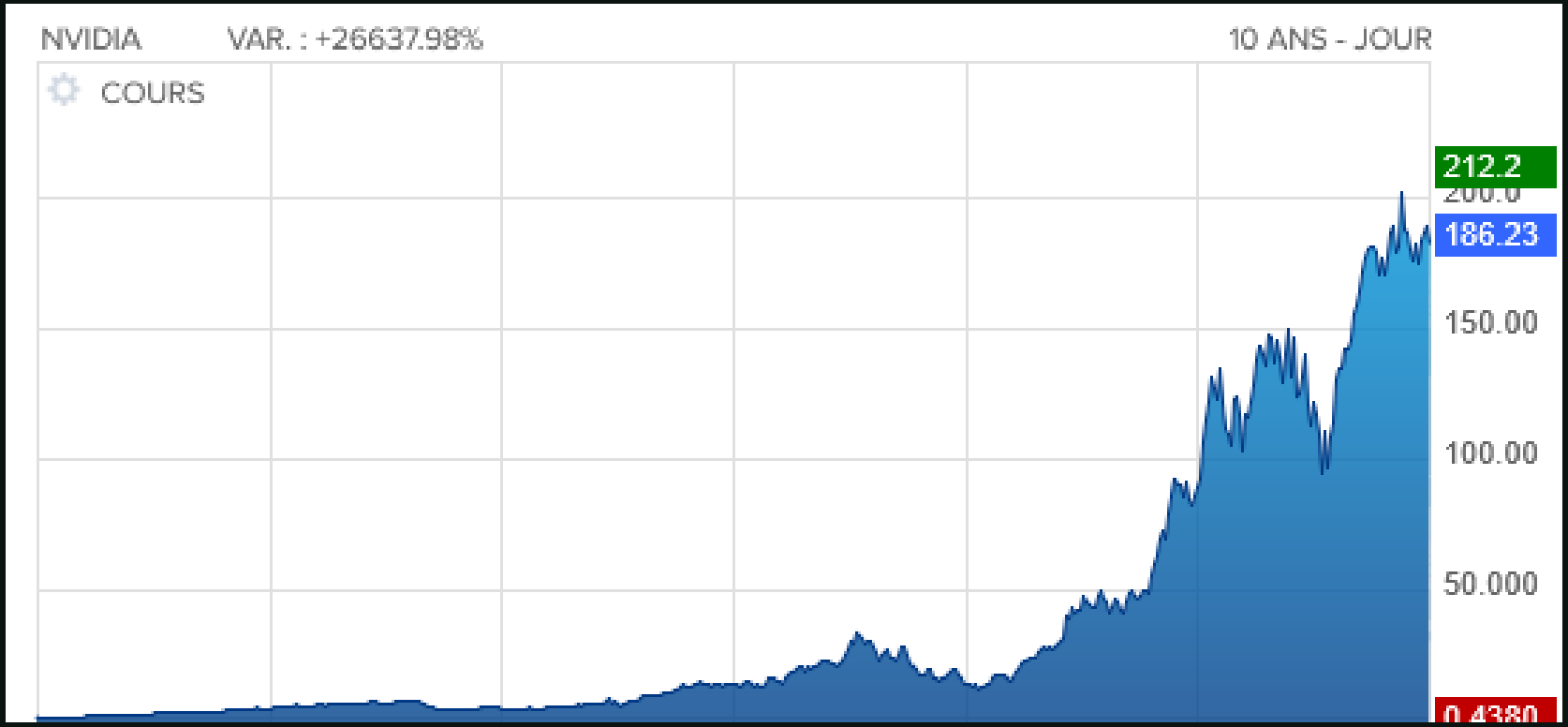
Revenu NVIDIA



https://blogs.alphanso.ai/blogs/the-remarkable-rise-of-nvidia/

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Cours action NVIDIA



https://www.boursorama.com/cours/NVDA/

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Introduction aux GPGPU

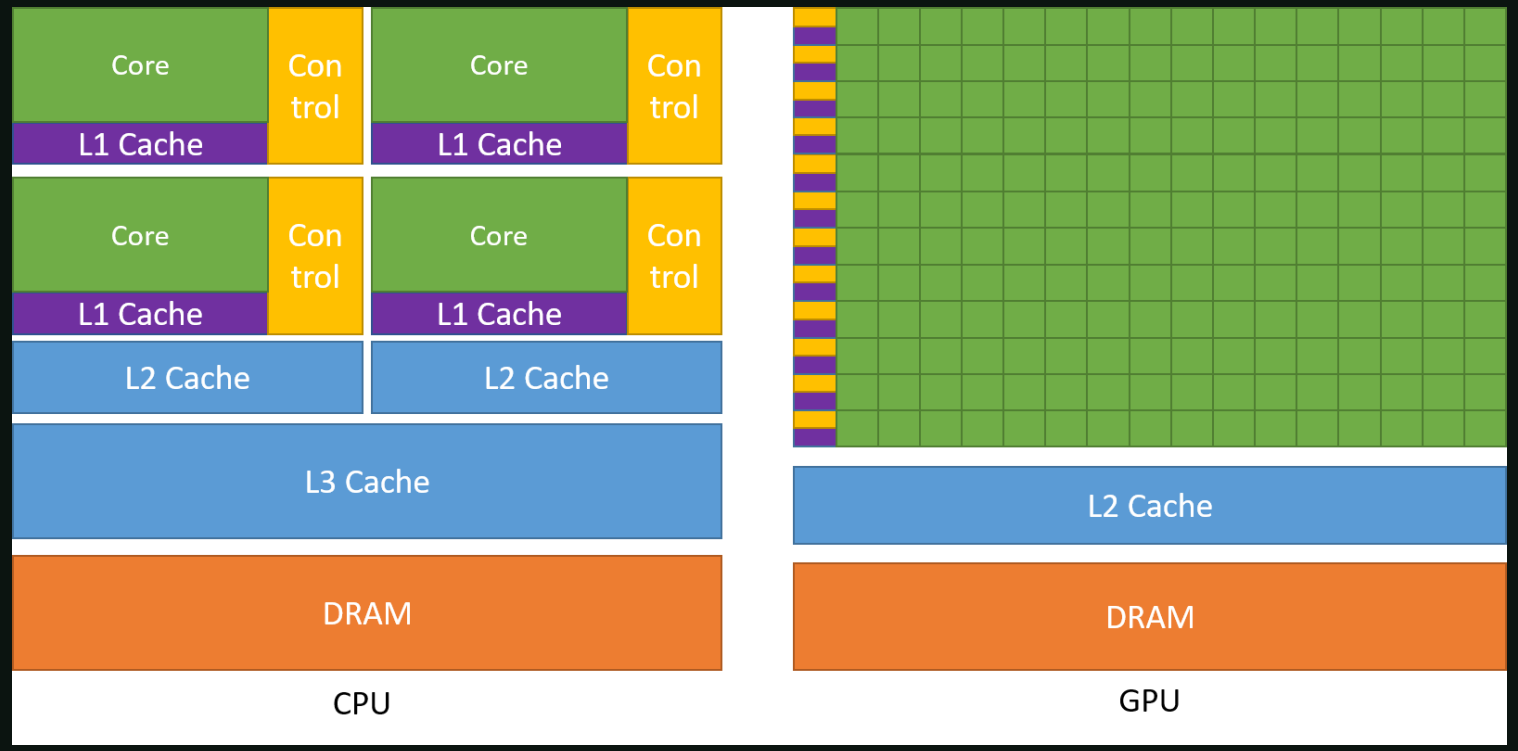
Historique des GPU

- Manque de puissance sur les PC & partie rendu graphique de plus en plus gourmande
- Objectifs :
  - Déléguer au GPU des tâches importantes (en termes de temps de calcul) mais répétitives
  - Effectuer ces taches bien plus rapidement qu’avec un CPU + libérer le CPU
- Méthodes :
  - Optimiser pour réaliser des calculs en virgule flottante
  - Optimiser pour réaliser la même opération sur des données différentes

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Principales différences architecturales des CPU et des GPU

- CPU :
  - + de contrôles
  - + de caches
- GPU :
  - + d'unités de calcul



https://docs.nvidia.com/cuda/cuda-c-programming-guide/

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

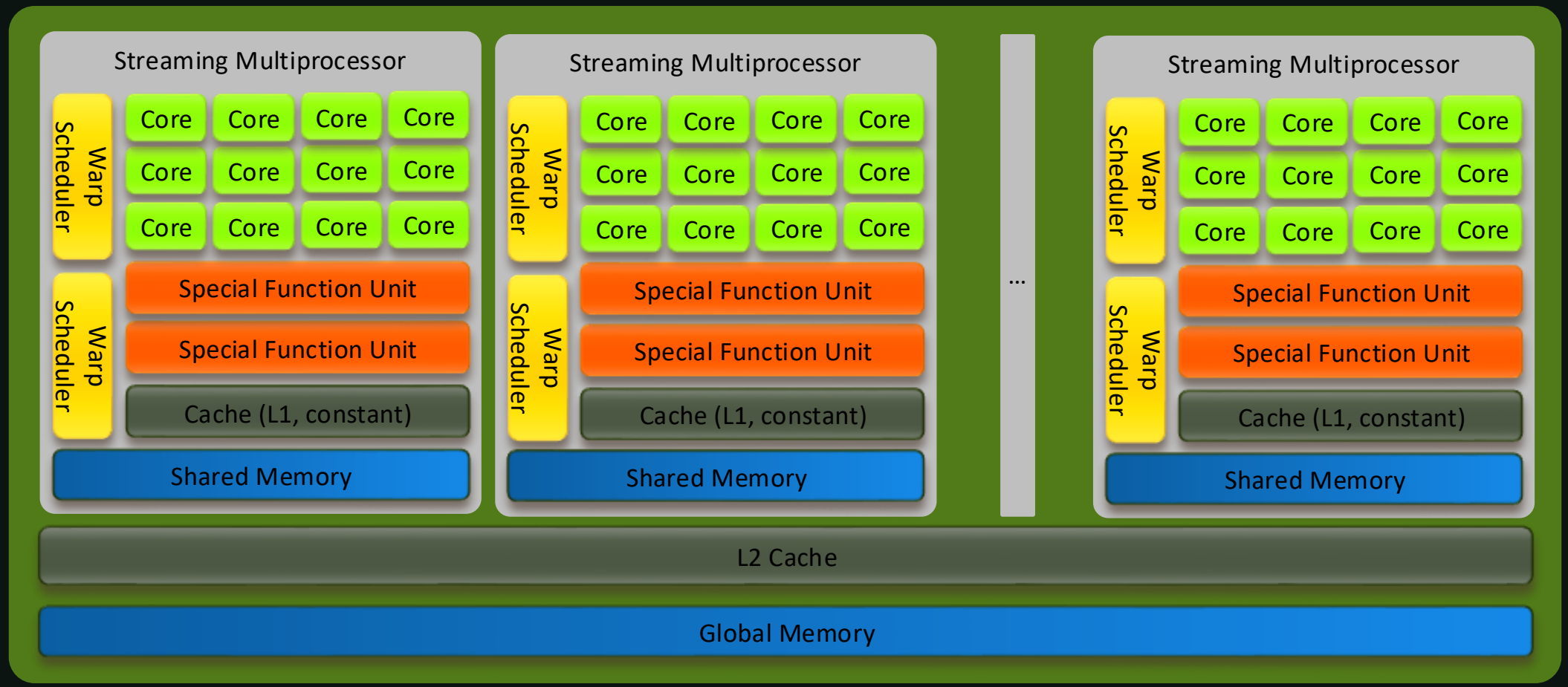
Impacts des différences architecturales sur la prog’ GPGPU

- Plus d’unités de calcul => Capacité de calcul brut plus élevés
- Moins de contrôles => Moins de facilité à effectuer des tâches diverses
- Moins de caches => Accès mémoire plus lents

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Architecture simplifiée d’un GPGPU 1/2

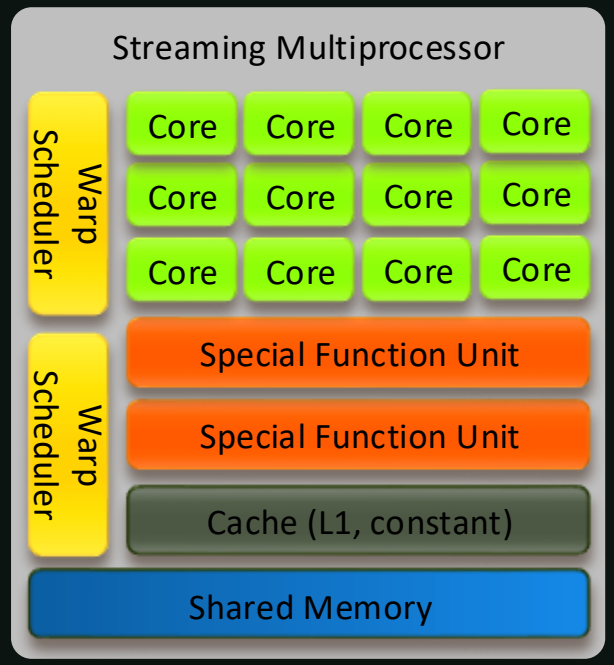
- /!\ Dépend des « Compute Capability » du GPU ⇔ de sa génération



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Architecture simplifiée d'un GPGPU 2/2

- Core — Cœur de calcul arithmétique
- Special Function Unit — Cœur de calcul des fonctions transcendante (cos, sin…, exp…)
- Warp Scheduler — Unité de contrôle
- Cache (L1, constant) — Cache le plus performant
- Shared Memory — Mémoire R/W la plus performante
- L2 Cache — Cache global au GPU
- Global Memory — Mémoire R/W la plus large



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple de génération de carte NVIDIA

- 5.X :
  - 128 Core
  - 32 SFU
  - 4 Warp Scheduler
  - 24KB Cache L1/constant
  - 64/96KB Shared Memory
  - L2 Cache et Global Memory dépendent du modèle
    - Exemple Tesla M40 : 3MB de cache L2 et 12Go de mémoire globale

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Résumé

- Beaucoup de puissance de calcul
- Pas beaucoup de cache
  - => Accès mémoire potentiellement complexe
- Pas beaucoup de contrôle
  - => Limité le changement de contexte

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Approche SIMD

Architecture de calcul parallèle

- De nombreuses architectures différentes existent :
  - SMP (Symmetric MultiProcessing) : au moins deux processeurs identiques connectés à une même mémoire
  - Cluster (ferme de calcul) : groupe de composants de calcul localement interconnectés entre eux
  - Grille de calcul : groupe de composants de calcul interconnectés entre eux pouvant être très fortement éloignés géographiquement
  - FPGA (Field-Programmable Gate Array) : circuit intégré programmable permettant la réalisation de calcul parallèle
  - GP-GPU

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Classification des architectures d’ordinateurs

- La taxonomie de Flynn propose une classification en quatre catégories :
  - SISD (Single Instruction, Single Data stream)

    Ex : Processeur mono-coeur traditionnel.
  - SIMD (Single Instruction, Multiple Data streams)

    Ex : GPU.
  - MISD (Multiple Instruction, Single Data stream)

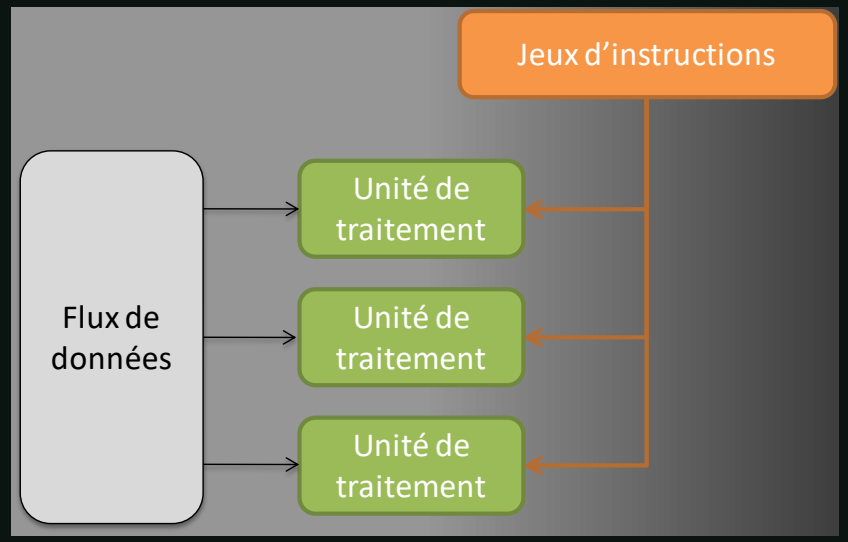
    Ex : Utilisé pour la détection d’erreur dans des applications très sensible.
  - MIMD (Multiple Instruction, Multiple Data streams)

    Ex : SMP, Cluster ou Grille de calcul.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Single Instruction, Multiple Data streams

- Exécution de la même instruction en parallèle sur différents flux de données
- Également utilisé dans les instructions SSE (Streaming SIMD Extensions) des processeurs actuels



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple de pseudo-code SIMD

```
int     vecteur_A[ 4 ] = { 2, 4, 6, 8 };
int     vecteur_B[ 4 ] = { 2, 3, 5, 7 };
int     vecteur_C[ 4 ];

// Sur une architecture non SIMD :
vecteur_C[ 0 ] = vecteur_A[ 0 ] + vecteur_B[ 0 ]; // 1er cycle
vecteur_C[ 1 ] = vecteur_A[ 1 ] + vecteur_B[ 1 ]; // 2ème cycle
vecteur_C[ 2 ] = vecteur_A[ 2 ] + vecteur_B[ 2 ]; // 3ème cycle
vecteur_C[ 3 ] = vecteur_A[ 3 ] + vecteur_B[ 3 ]; // 4ème cycle

// Sur une architecture SIMD avec quatre unités de traitement,
// chaque unité de traitement effectue en parallèle :
vecteur_C[ numero_unite ] = vecteur_A[ numero_unite ] +
                            vecteur_B[ numero_unite ]; // 1er et unique cycle
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

SIMD pour les GP-GPU : SIMT

- NVIDIA parle de Single Instruction, Multiple Threads (SIMT)
- Exécution de plusieurs threads en parallèle exécutant le même code mais traitant des données différentes
- Chaque thread possède un identifiant unique lui permettant d’accéder à une donnée propre

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Gestion des threads – les kernels

Introduction sur les kernels

- Kernel : fonction invoquée par l'hôte (le CPU) et exécutée plusieurs fois en parallèle sur le GPU
  - ⇔ Fonctions sur GPU
- Paramétré lors de l'appel par le nombre de threads utilisés (en plus des paramètres habituels)
- Accède à la mémoire du GPU uniquement
  - => Impossibilité d'accéder à la mémoire du CPU

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Appel de kernel 1/2

```
__global__ void monPremierKernel()
{
}

int main( int, char *[] )
{
    monPremierKernel<<< 1, 10 >>>();
}
```

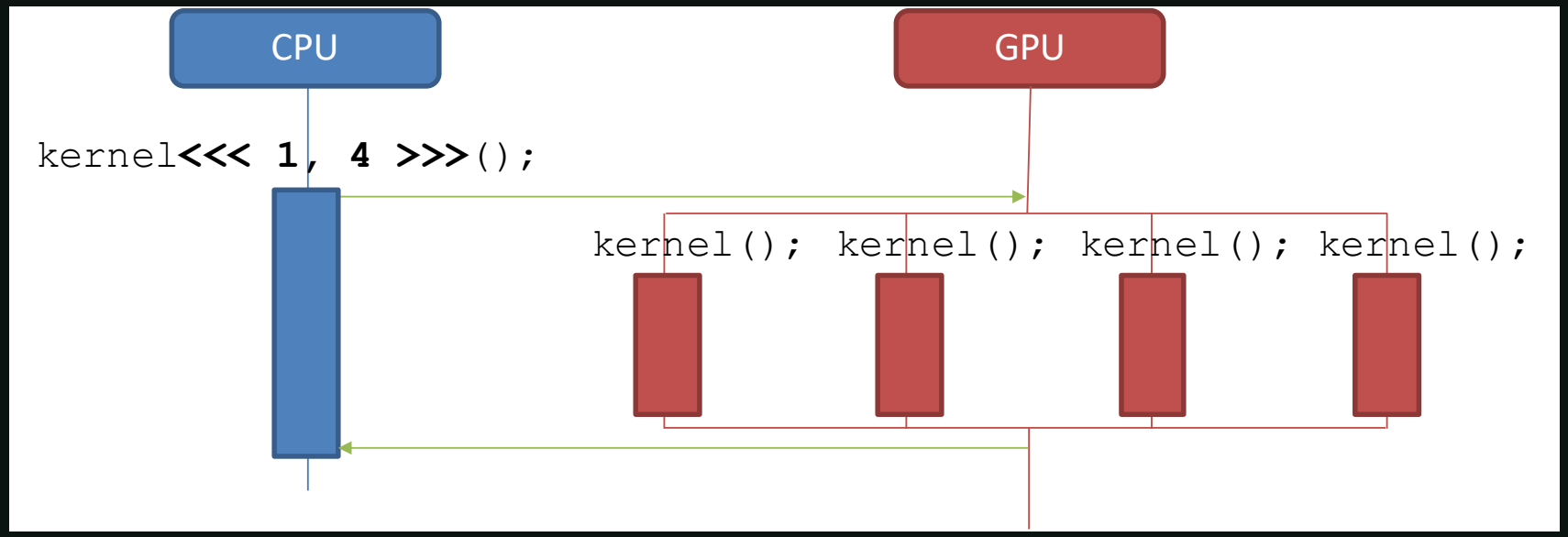
- Appel standard (simplifié) d'un kernel :

```
monKernel<<< nbBloc, nbThdParBloc >>>(
                                      /*paramètres*/
                                     );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Appel de kernel 2/2

```
__global__ void kernel()
{}
int main( int, char *[] )
{
    kernel<<< 1, 4 >>>();
}
```



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Déclaration d’un kernel

- Mot clé CUDA : `__global__`.
  - Spécifie que la fonction est un kernel.
    - Appelé depuis l’hôte (le CPU).
    - Exécuté sur le GPU.
- Mot clé CUDA : `__device__`.
  - Appelé et exécuté sur le GPU.
- Exemple :

```
__global__ void multiMatrice( float * inMatrice_A,
                              float * inMatrice_B,
                              float * outMatrice_C );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Notion de warp 1/2

- Notion logicielle utilisée par NVIDIA et qui correspond au regroupement de 32 threads « identiques »
- Exécution obligatoire d’un nombre rond de warp
  - Dans le cas contraire, arrondi au nombre de warp supérieur
- Exécution sur le GPU par warp (par groupe de 32 threads)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Notion de warp 2/2

- Tous les threads d'un warp joue le même jeu d'instruction (SIMT)
  - => Si flux d'instructions différents, cycle différents

```
// Mult x2 + 1 si data pair ou + 2 si impair
res = data[threadId] * 2;
if (data[threadId]%2 == 0) {
  res += 1; // certains threads
} else {
  res += 2; // les autres threads
}
```

- Impact important sur le comportement du GPU
  - => Impact tout aussi important sur l'implémentation
  - Nouvelles archi' GPU de plus en plus permissives

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Gestion des threads – les blocs

Introduction sur les blocs

- Regroupement sous forme de blocs des threads exécutés sur le GPU
- « Exécution simultanée » des threads d'un même bloc
  - Possibilités de communication entre ces threads
  - Possibilités de partage de mémoire entre ces threads
- Limitation matérielle du nombre de threads par bloc
  - Évolue maintenant peu avec les versions de GPU (1024 depuis près de 10 ans)
  - Du à la mémoire nécessaire pour faire du scheduling de threads

Dimension des blocs utilisés

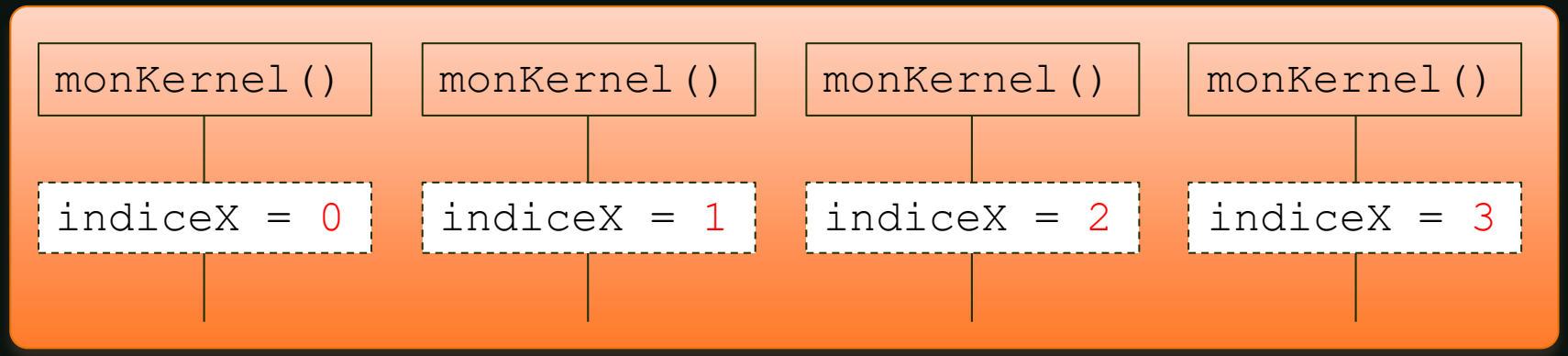
- `kernel<<< 1, 10 >>>();`

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Différenciation des threads d'un bloc : threadIdx

- Sans différenciation des threads, travail similaire effectué par tous les threads
  - threadIdx : variable CUDA identifiant de manière unique un thread dans un bloc

```
__global__ void monKernel() { int indiceX  = threadIdx.x; }

monKernel<<< 1, 4 >>>();
```

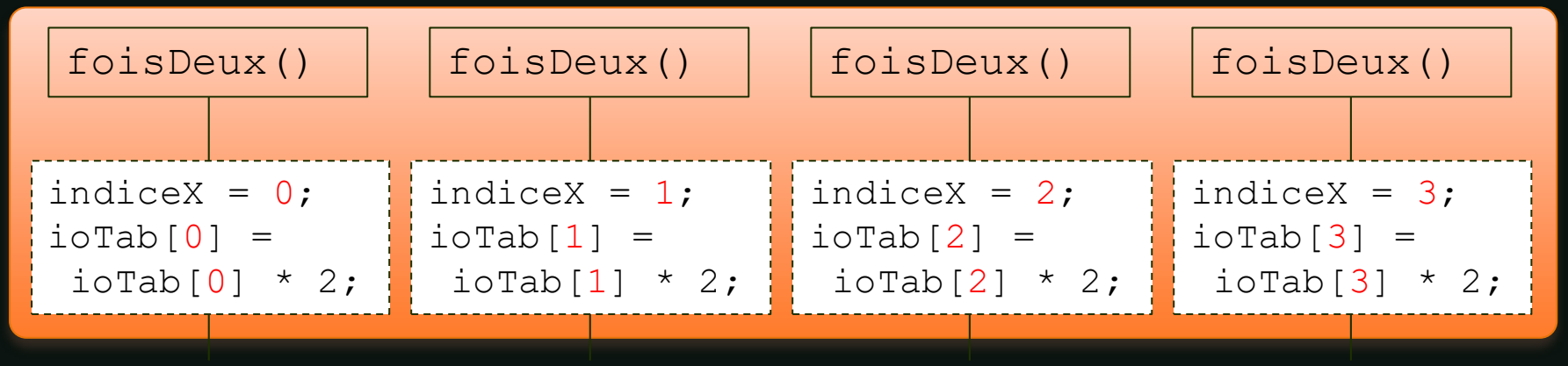


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Première application « utile »

```
__global__ void foisDeux( float * inoutTab )
{
   int indiceX  = threadIdx.x;
   inoutTab[ indiceX ] = inoutTab[ indiceX ] * 2;
}

float tableau[] = { 2, 4, 6, 8 };
foisDeux<<< 1, 4 >>>( tableau );
```

/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU + overflow



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Premier exercice : addition de vecteur

```
__global__ void ajout_vecteur(                )
{
   // Calcul de l'indice
   int idX  =            ;
   // Réalisation de l'ajout
                                  
}

int main( int, char *[] )
{
   float tableau_A[ 5 ] = { 2, 4, 6, 8, 10 };
   float tableau_B[ 5 ] = { 1, 2, 3, 4, 5 };
   float tableau_C[ 5 ];

   // Appel du kernel
                                  
}
```

/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU + overflow

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Premier exercice : correction

```
__global__ void ajout_vecteur( float * inTabA, float * inTabB, float * outTabC )
{
   int idX  = threadIdx.x;
   outTabC[ idX ] = inTabA[ idX ] + inTabB[ idX ];
}

int main( int, char *[] )
{
   float tableau_A[ 5 ] = { 2, 4, 6, 8, 10 };
   float tableau_B[ 5 ] = { 1, 2, 3, 4, 5 };
   float tableau_C[ 5 ];

   ajout_vecteur<<< 1, 5 >>>( tableau_A, tableau_B, tableau_C );
}
```

/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU + overflow

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Bloc à plusieurs dimensions 1/3

- Prototype (presque exact) d'un appel de kernel :

```
nom_du_kernel<<< dim3, dim3 >>>( [parametres] );
```

- dim3, type basé sur uint3 :

```
struct uint3 { unsigned int x, y, z; };
```

- Pour chaque dimension :
  - Valeur par défaut : 1
  - Valeur maximale : dépend des versions (1024 pour les dernières générations)
- Ex :

```
kernel<<< 1, 10 >>>() ⇔ kernel<<< 1, dim3( 10, 1, 1 ) >>>();
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

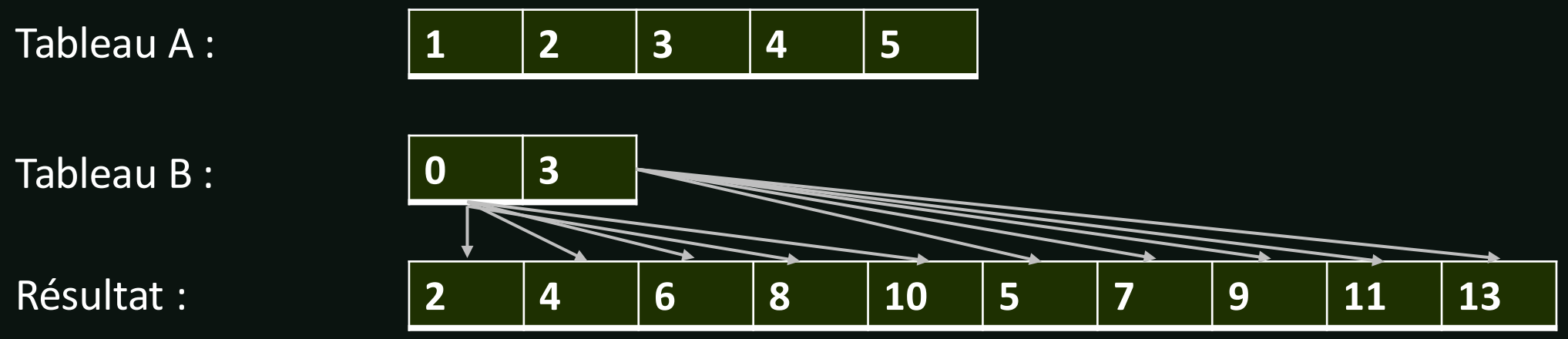
Bloc à plusieurs dimensions 2/3

- blockDim : fournit, à l’exécution, les dimensions du bloc dans lequel le kernel est exécuté
- Fonctionnement similaire à threadIdx :
  - Également de type dim3
  - Accès à la taille de chaque dimension à l’aide des composantes x, y et z

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Bloc à plusieurs dimensions 3/3

- Multiplication d’un tableau par 2 + ajout de chaque valeur d’un second tableau
- N valeurs au départ => nb de valeurs du second tableau * N valeurs à l’arrivée



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Bloc à plusieurs dimensions 3/3

```
__global__ void tabAfoisDeuxPlusTabB( float * inTabA, float * inTabB, float * outTabC ) {
   int idX      = threadIdx.x;
   int idY      = threadIdx.y;
   int idGlobal = idX + idY * blockDim.x;

   outTabC[ idGlobal ] = inTabA[ idX ] * 2 + inTabB[ idY ];
}

int main( int, char *[] ) {
   float tabA[ 5 ] = { 1, 2, 3, 4, 5 };
   float tabB[ 2 ] = { 0, 3 };
   float tabC[ 10 ];

   dim3 dimBloc( 5, 2 );
   tabAfoisDeuxPlusTabB<<< 1, dimBloc >>>( tabA, tabB, tabC );
}
```

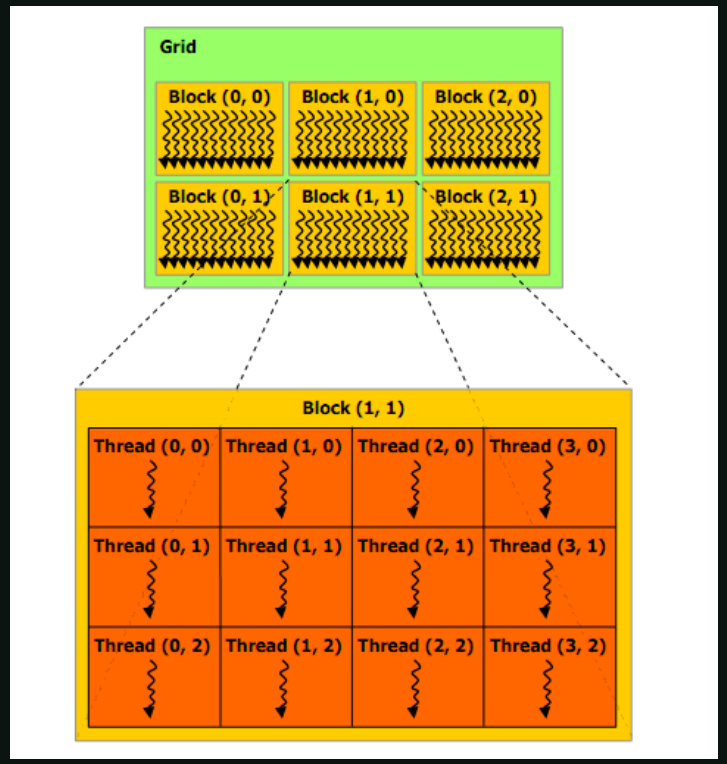
/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU + overflow

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Gestion des threads – la grille

Introduction

- Nombre total de threads par bloc limité (1024 sur les GPU actuels)
  - Très limité en comparaison de la capacité de calcul en parallèle
  - => Utilisation d'une grille contenant plusieurs blocs de dimension identique
- Dimension maximale pour un GPU récent :
  - De la grille : en 'x' : 2^{31}-1; en 'y' et 'z' : 65535
  - D'un bloc : en 'x' et 'y' : 1024'; en 'z' : 64
  - Nombre maximum de threads par bloc : 1024
  - => Jusqu'à plus de $9x10^{21}$ threads.



Source NVIDIA

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

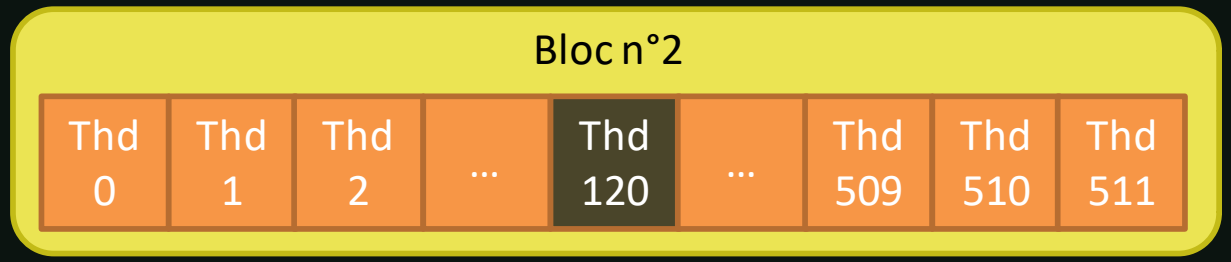
blockIdx et gridDim

- blockIdx : à l’exécution, dans un kernel, position du bloc auquel le thread appartient
  - Fonctionnement similaire à threadIdx
- gridDim : à l’exécution, dans un kernel, dimension de la grille
  - Fonctionnement similaire à blocDim.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Multiplication de grands tableaux

```
__global__ void doubleGrandTableau( float * inoutTab )
{
   int idX   = threadIdx.x + blockIdx.x * blockDim.x;
   inoutTab[ idX ] = inoutTab[ idX ] * 2;
}
```




```
threadIdx.x  = 120
blockIdx.x   = 2
blockDim.x   = 512
idx = 120 + 2 * 512 = 1144
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

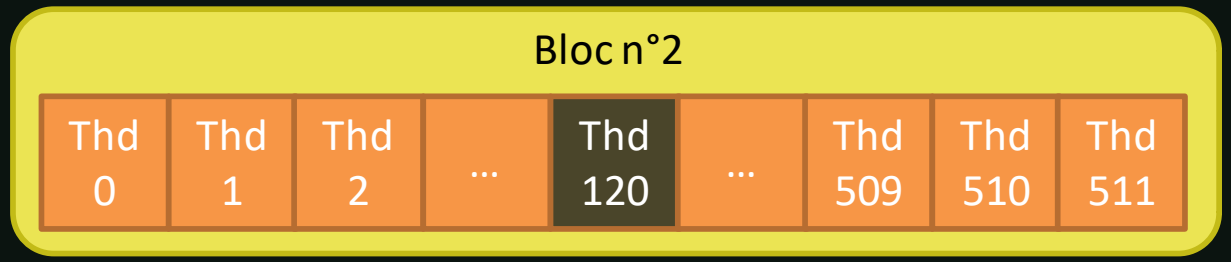
Rappel

- __global__ + fonction => kernel
- dim3 : champs x, y et z disponible
- threadIdx : indice du thread dans le bloc
- blockIdx : indice du bloc dans la grille
- blockDim : dimension des blocs
- gridDim : dimension de la grille

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Rappel

```
__global__ void doubleGrandTableau( float * inoutTab )
{
   int idX   = threadIdx.x + blockIdx.x * blockDim.x;
   inoutTab[ idX ] = inoutTab[ idX ] * 2;
}
```




```
threadIdx.x  = 120
blockIdx.x   = 2
blockDim.x   = 512
idx = 120 + 2 * 512 = 1144
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Deuxième exercice : utilisation de grilles à deux dimensions

```
__global__ void doubleImmenseTab( float * inoutTab )
{
   int idX       =                 ;
   int idY       =                 ;
   int idGlobal  =                 ;
   inoutTab[ idGlobal ] = inoutTab[ idGlobal ] * 2;
}
int main( int, char *[] )
{
   dim3  dimGrille( 1000, 1000 );
   dim3  dimBloc( 512, 2 );
   float tab[ 1024000000 ];
   // Initialisation du tableau
   [...]
   doubleImmenseTab<<< dimGrille, dimBloc >>>( tab );
}
```

/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Deuxième exercice : correction

```
__global__ void doubleImmenseTab( float * inoutTab )
{
   int idX       = threadIdx.x + blockIdx.x * blockDim.x;
   int idY       = threadIdx.y + blockIdx.y * blockDim.y;
   int idGlobal  = idX + blockDim.x * gridDim.x * idY;
   inoutTab[ idGlobal ] = inoutTab[ idGlobal ] * 2;
}
int main( int, char *[] )
{
   dim3  dimGrille( 1000, 1000 );
   dim3  dimBloc( 512, 2 );
   float tab[ 1024000000 ];
   // Initialisation du tableau
   [...]
   doubleImmenseTab<<< dimGrille, dimBloc >>>( tab );
}
```

/!\ Code incorrect
utilisation de la mémoire CPU sur le GPU

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Blocs et grilles à multiple dimensions

- Nécessaire dans les premières versions de CUDA pour manipuler de grands indices
- Nécessaire maintenant pour manipuler de très grands indices
- Toujours pratique pour manipuler des tableaux à deux dimensions ou faire deux traitements transversaux

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

TD1 – calcul $nx^2 + my$

- TD car on ne gère pas la mémoire pour le moment
- Proposer le code pour calculer sur GPU :
  - L'ensemble des n valeurs $nx^2 + my$
  - Où 'n' et 'm' sont saisis par l'utilisateur
  - Pour un très grand nombre de valeurs (taille des tableaux 'tabX' et 'tabY' > 1000000)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

TD2 – conversion RGB vers niveau de gris

- TD car on ne gère pas la mémoire pour le moment
- Proposer le code pour calculer sur GPU :
  - La conversion de l'ensemble des pixels de l'image de RGB vers niveau de gris (0,299xR + 0,587xG + 0,114xB)
  - Image d'entrée et de sortie, tableau de char (1 octet pour coder de 0 à 255 R, G ou B) à trois dimensions
    - 1ère pour la dimension x
    - 2ème pour la dimension y
    - 3ème pour les valeurs R, G et B

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémo

- __global__ + fonction => kernel
- dim3 : champs x, y et z disponible
- threadIdx : indice du thread dans le bloc
- blockIdx : indice du bloc dans la grille
- blockDim : dimension des blocs
- gridDim : dimension de la grille
- TD1 : nx^2 + my avec n et m saisie par l'utilisateur
- TD2 : RGB vers niveau de gris (0,299xR + 0,587xG + 0,114xB)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémoires

Introduction à la mémoire

- Impossibilité d’accéder depuis le GPU à la mémoire du CPU
  - Nécessité d’utiliser la mémoire du GPU pour les (grands) tableaux impossible à passer valeur par valeur en paramètre des kernels
- Plusieurs niveaux de mémoire dans les GPU :
  - Propre à un thread. Ex : les registres
  - Commun à l’ensemble des threads d’un même bloc. Ex : la mémoire partagée
  - Commun à l’ensemble des threads de la grille. Ex : la mémoire globale
- Temps de latence décroit avec la visibilité
- Possibilité d’utiliser des caches en taille limitée

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémoire globale

Introduction à la mémoire globale

- Impossibilité d’accéder depuis le GPU à la mémoire du CPU
  - Nécessité de réaliser des copies sur le GPU des tableaux à traiter
  - Utilisation de la mémoire globale
- Caractéristiques :
  - Présente en grande quantité (plusieurs dizaines de Go sur les plus grosses cartes)
  - Visibilité à l’ensemble des threads de la grille
  - Persistance entre les appels de kernel
  - Très très importante latence (de l’ordre de la centaine de cycles)

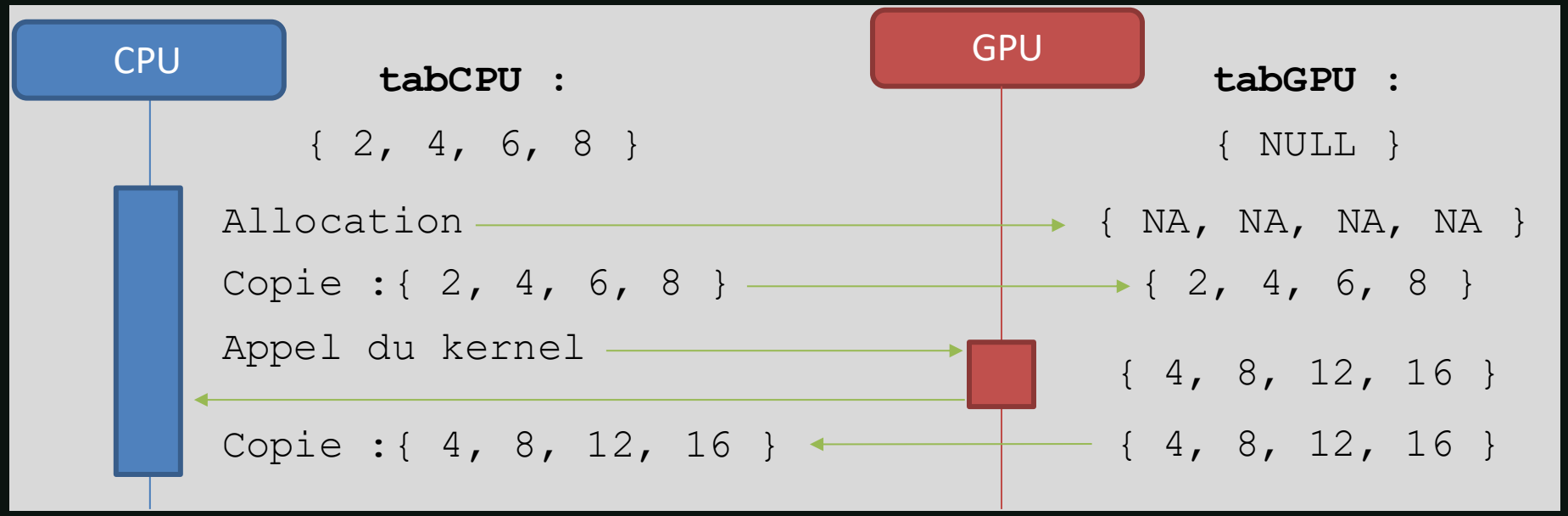
© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Principe de base de l’utilisation de la mémoire globale

- Utilisée pour le stockage des grands tableaux.
- Algorithme classique d’exécution d’un kernel traitant un tableau tab_CPU de taille taille_tab :
  1. Allocation d’un tableaux de taille équivalente (taille_tab) sur le GPU (tab_GPU)
  2. Copie des données du tableau présent sur le CPU (tab_CPU) vers celui présent sur le GPU (tab_GPU)
  3. Appel du kernel sur le tableau présent sur le GPU (tab_GPU)
  4. Copie des données du tableau du GPU (tab_GPU), mis à jour par le kernel, vers celui du CPU(tab_CPU)

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Représentation schématique de l'utilisation de la mémoire globale



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Allocation de la mémoire

- Allocation réalisée de manière semblable au malloc du C.
  - Prend en paramètre le nombre d'**octets** à allouer et pas le nombre d'éléments du tableau comme le new du C++
- Prototype :

```
cudaError_t cudaMalloc( void **   inPtrAdr,
                        size_t    inNbOctets );
```

- À la fin de l'appel, inAdrPtr pointe sur l'adresse du tableau sur le GPU

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Allocation de la mémoire

- Prototype :

```
cudaError_t cudaMalloc( void **   inPtrAdr,
                        size_t    inNbOctets );
```

- Ex :

```
int       tailleTab = 100;
float *   tab_GPU;
cudaMalloc( &tab_GPU, tailleTab * sizeof( float ) );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Copie de la mémoire CPU->GPU et GPU->CPU

- Nécessite de passer le nombre d'octets à copier
  - Source d'erreurs pas toujours simple à repérer !
- Fonction identique utilisée pour la copie du CPU vers le GPU et du GPU vers le CPU.
  - Sens de la copie spécifié à l'aide des valeurs :
    - cudaMemcpyHostToDevice : CPU vers GPU.
    - cudaMemcpyDeviceToHost : GPU vers CPU.
  - Source d'erreurs pas toujours simple à repérer !
- Prototype simplifié :

```
cudaMemcpy( dest, src, tailleEnOctets, sensDeCopie )
```

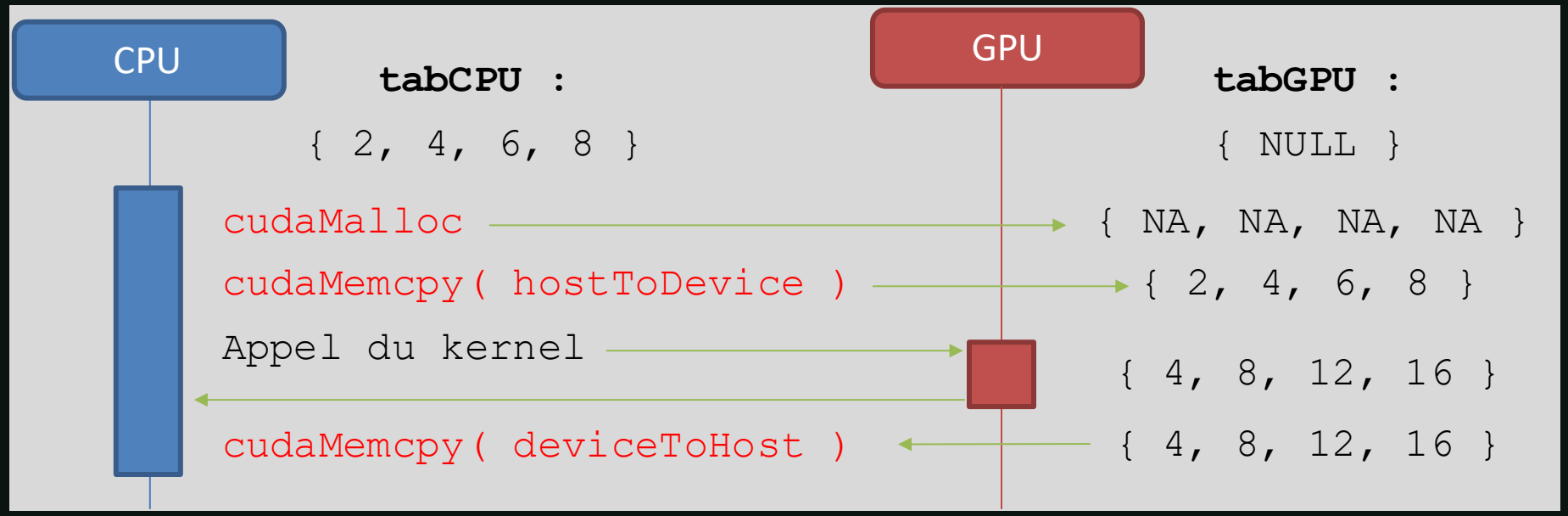

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple de copie aller et retour

```
int       tailleTab = 100;
float *   tab_CPU;
float *   tab_GPU;

tabCPU = new float[ tailleTab ];
cudaMalloc( &tab_GPU, tailleTab * sizeof( float ) );

[...] // Initialisation du tableau sur le CPU

// Copie aller :
cudaMemcpy( tab_GPU, tab_CPU, tailleTab * sizeof( float ),
            cudaMemcpyHostToDevice );

[...] // Appel du kernel

// Copie retour :
cudaMemcpy( tab_CPU, tab_GPU, tailleTab * sizeof( float ),
            cudaMemcpyDeviceToHost );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Représentation schématique de l'utilisation de la mémoire globale (2ème !)



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Premier exercice qui compile et fonctionne

```
__global__ void ajoutVecteurs( float * inA, float * inB, float * outC,
                               int inTaille ) {
   int indiceX  = threadIdx.x + blockIdx.x * blockDim.x;
   if ( indiceX < inTaille )
      outC[ indiceX ] = inA[ indiceX ] + inB[ indiceX ];
}

float * ajoutVecteurSurGPU( float * inA, float * inB,
                            const int inTaille ) {
   dim3  dimBloc( 512 );
   dim3  dimGrille( calcul du nombre de blocs nécessaires );
   Variables utiles

   Allocations & Copies aller

   ajoutVecteurs <<< dimGrille, dimBloc >>>( paramètres );

   Copie retour
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Calcul du nombre de blocs nécessaires

```
float * ajoutVecteurSurGPU( float * inA, float * inB, const int inTaille )
{
   dim3  dimBloc( 512 );
   dim3  dimGrille( inTaille / dimBloc.x +
                    ( inTaille % dimBloc.x != 0 ) );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Variables utiles + Allocations & Copies aller

```
float * ajoutVecteurSurGPU( float * inA, float * inB, const int inTaille )
{
   ...
   float * outC;
   float * A_GPU;
   float * B_GPU;
   float * C_GPU;

   outC = new float[ inTaille ];
   cudaMalloc( &A_GPU, inTaille * sizeof( float ) );
   cudaMalloc( &B_GPU, inTaille * sizeof( float ) );
   cudaMalloc( &C_GPU, inTaille * sizeof( float ) );

   cudaMemcpy( A_GPU, inA, inTaille * sizeof( float ),
               cudaMemcpyHostToDevice );
   cudaMemcpy( B_GPU, inB, inTaille * sizeof( float ),
               cudaMemcpyHostToDevice );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Copie retour

```
float * ajoutVecteurSurGPU( float * inA, float * inB, const int inTaille )
{
   ...
   ajoutVecteurs <<< dimGrille, dimBloc >>>( A_GPU, B_GPU, C_GPU, inTaille );

   cudaMemcpy( outC, C_GPU, inTaille * sizeof( float ), cudaMemcpyDeviceToHost );

   return outC;
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

```
__global__ void ajoutVecteurs( float * inA, float * inB, float * outC,
                               int inTaille )
{
   int indiceX  = threadIdx.x + blockIdx.x * blockDim.x;
   if ( indiceX < inTaille )
      outC[ indiceX ] = inA[ indiceX ] + inB[ indiceX ];
}

float * ajoutVecteurSurGPU( float * inA, float * inB, const int inTaille )
{
   dim3  dimBloc( 512 );
   dim3  dimGrille( inTaille / dimBloc.x +
                    ( inTaille % dimBloc.x != 0 ) );
   float * outC;
   float * A_GPU;
   float * B_GPU;
   float * C_GPU;

   outC = new float[ inTaille ];
   cudaMalloc( &A_GPU, inTaille * sizeof( float ) );
   cudaMalloc( &B_GPU, inTaille * sizeof( float ) );
   cudaMalloc( &C_GPU, inTaille * sizeof( float ) );

   cudaMemcpy( A_GPU, inA, inTaille * sizeof( float ),
               cudaMemcpyHostToDevice );
   cudaMemcpy( B_GPU, inB, inTaille * sizeof( float ),
               cudaMemcpyHostToDevice );

   ajoutVecteurs <<< dimGrille, dimBloc >>>( A_GPU, B_GPU, C_GPU, inTaille );

   cudaMemcpy( outC, C_GPU, inTaille * sizeof( float ), cudaMemcpyDeviceToHost );

   return outC;
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémoire 2D et 3D

- Également possible d’utiliser des fonctions dédiées pour les tableaux 2D et 3D
  - Gère le padding automatiquement
- cudaMallocPitch et cudaMalloc3D pour l’allocation
- cudaMemcpy2D et cudaMemcpy3D pour la copie

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Problème de la mémoire globale

- Énormes temps de latence (de l’ordre de plusieurs centaines de cycles)
  - Masqués autant que possible par l’ordonnancement des threads
  - De plus en plus de cache disponible pour masquer
- Jusqu’à 32 fois plus lente en cas de lecture non coalescente
- Autres types de mémoire plus rapide à utiliser dès que possible
- Obligatoire pour les « communications » au sein de l’ensemble des threads de la grille

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Lecture coalescente

- ≈ Lecture de façon contigüe => important de bien calculé l'index du thread
- Chargement de la mémoire par warp (un warp contient 32 threads)
- Dépend des compute capabilities
- Transaction de 32, 64 ou 128 octets
  - Chargement de plus de 4 octets par thread => plusieurs transactions
  - Aligner la taille des structures (__align__)

```
struct __align__(16) {
   float x;
   float y;
   float z;
};
```

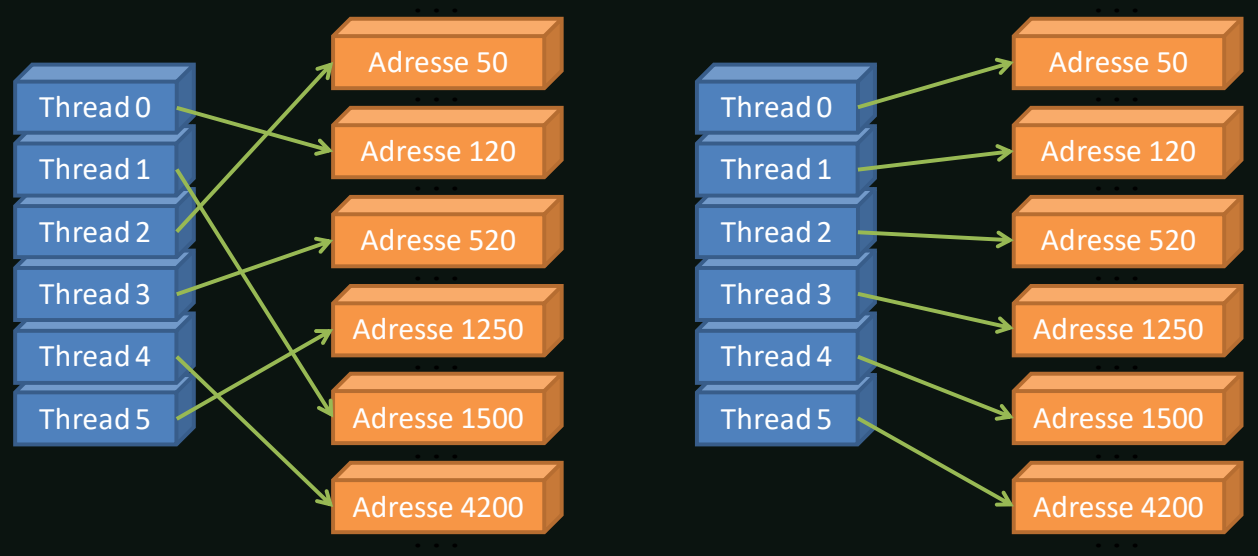

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemples de lectures non coalescente et de lectures coalescente

Lecture

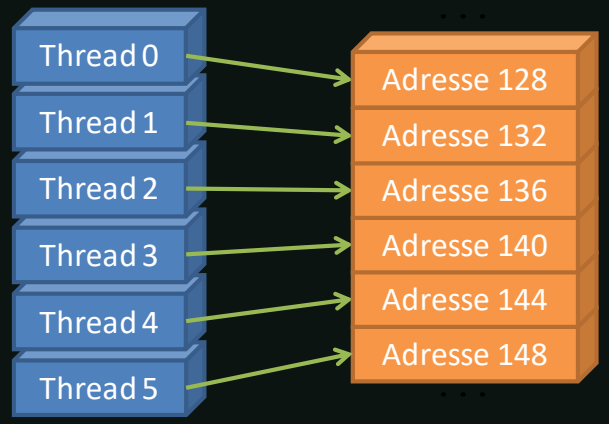
NON

coalescente



Lecture

coalescente



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

TP 1

- Revenir sur les TD pour gérer la mémoire

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

TP 2

- Proposer un programme permettant de ne conserver que les nombres premiers d’une liste
- Sur CPU :
  - Crible d'Ératosthène pour générer, sur CPU, les nombres premiers entre 1 et NB_MAX
  - Remplir un tableau de nombres pseudo-aléatoire (https://cplusplus.com/reference/random/)
- Sur GPU :
  - Identifier et « supprimer » les nombres non premiers

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Algorithme de vérification de la primalité d’un nombre (version mémoire globale)

```
Calcul de l’indice global.

Si le thread doit effectuer des calculs faire

  Pour chaque nombre premier de la liste en mémoire globale faire

    Si le nombre aléatoire est multiple de ce nombre alors

      Le marquer comme tel en mémoire globale
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Code du kernel vérifiant la primalité d'un nombre (version mémoire globale)

```
__global__ void filtreNbPremiers(       int * inoutListeNombresAleatoires,
                                  const int   inNbNombresAleatoires,
                                  const int * inListeNombresPremiers,
                                  const int   inNbNombresPremiers )
{
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( indiceGlobal < inNbNombresAleatoires )
   {
      float   nombreAleatoire    = inoutListeNombresAleatoires[ indiceGlobal ];

      int     i                  = 0;
      while ( i < inNbNombresPremiers )
      {
         if (nombreAleatoire % inListeNombresPremiers[ i ] == 0 )
         {
            inoutListeNombresAleatoires[ indiceGlobal ] = -1;
            break;
         }
         i++;
      }
   }
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Algorithme de vérification de la primalité d’un nombre (version mémoire globale)

Adobe

```
Calcul de l’indice global.

Si le thread doit effectuer des calculs faire

  Récupérer en mémoire globale le nombre aléatoire à tester.

  Considérer que le nombre est premier.

  Pour chaque nombre premier de la liste en mémoire globale faire

    Si le nombre aléatoire est multiple de ce nombre alors

      Le marquer comme tel en mémoire globale
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Code du kernel vérifiant la primalité d'un nombre (version mémoire globale)

```
__global__ void filtreNbPremiers(       int * inoutListeNombresAleatoires,
                                  const int   inNbNombresAleatoires,
                                  const int * inListeNombresPremiers,
                                  const int   inNbNombresPremiers )
{
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( indiceGlobal < inNbNombresAleatoires )
   {
      float   nombreAleatoire    = inoutListeNombresAleatoires[ indiceGlobal ];
      bool    estNombrePremier   = true;

      int     i                  = 0;
      while ( estNombrePremier && i < inNbNombresPremiers )
      {
         estNombrePremier   = nombreAleatoire % inListeNombresPremiers[ i ] == 0;
         i++;
      }

      if ( !estNombrePremier )
      {
         inoutListeNombresAleatoires[ indiceGlobal ] = -1;
      }
   }
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémoire partagée

Introduction à la mémoire partagée

- Pas de latence supplémentaire si accès coalescent
- Accessible en lecture et en écriture depuis un kernel s’exécutant sur le GPU
- Mémoire commune aux threads d’un bloc et seulement aux threads d’un bloc
- En quantité (très) limitée !

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Utilité de la mémoire partagée

- Pas accessible depuis le CPU
  - Impossible à utiliser en copiant les tableaux entre le CPU et le GPU puis en y accédant
- Utilisée pour diminuer les accès à la mémoire globale
  - Mutualisation de la lecture des valeurs lues de multiples fois en mémoire globale
- Ex : partager le tableau des nombres premiers dans le TP

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Code du kernel vérifiant la primalité d'un nombre (version mémoire globale)

```
__global__ void filtreNbPremiers(       int * inoutListeNombresAleatoires,
                                  const int   inNbNombresAleatoires,
                                  const int * inListeNombresPremiers,
                                  const int   inNbNombresPremiers )
{
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( indiceGlobal < inNbNombresAleatoires )
   {
      float   nombreAleatoire    = inoutListeNombresAleatoires[ indiceGlobal ];
      bool    estNombrePremier   = true;

      int     i                  = 0;
      while ( estNombrePremier && i < inNbNombresPremiers )
      {
         estNombrePremier   = nombreAleatoire % inListeNombresPremiers[ i ] == 0;
         i++;
      }

      if ( !estNombrePremier )
      {
         inoutListeNombresAleatoires[ indiceGlobal ] = -1;
      }
   }
}
```

Propre au thread et utilisé une fois en lecture et une fois en écriture
=> « Très bien » en mémoire globale

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Code du kernel vérifiant la primalité d'un nombre (version mémoire globale)

```
__global__ void filtreNbPremiers(       int * inoutListeNombresAleatoires,
                                  const int   inNbNombresAleatoires,
                                  const int * inListeNombresPremiers,
                                  const int   inNbNombresPremiers )
{
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( indiceGlobal < inNbNombresAleatoires )
   {
      float   nombreAleatoire    = inoutListeNombresAleatoires[ indiceGlobal ];
      bool    estNombrePremier   = true;

      int     i                  = 0;
      while ( estNombrePremier && i < inNbNombresPremiers )
      {
         estNombrePremier   = nombreAleatoire % inListeNombresPremiers[ i ] == 0;
         i++;
      }

      if ( !estNombrePremier )
      {
         inoutListeNombresAleatoires[ indiceGlobal ] = -1;
      }
   }
}
```

Accès identique pour chacun des threads
=> Mutualiser en mémoire partagée

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Idée générale de la lecture en mémoire partagée

Algorithme de lecture en mémoire partagée de N éléments nécessaire à l'ensemble des threads :

1. Les threads possédant un identifiant inférieur à N (et uniquement eux) : chargement d'un valeur depuis la mémoire globale vers la mémoire partagée
2. Synchronisation de l'ensemble des threads !
3. Réalisation de l'algorithme classique avec la lecture de certaines données depuis la mémoire partagée

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Algorithme de vérification de la primalité d'un nombre (version mémoire partagée)

Adobe

```
Calcul de l'indice global.
Si le thread doit charger la mémoire partagée faire
  Charger une valeur de la mémoire globale en mémoire partagée
Synchronisation des threads
Si le thread doit effectuer des calculs faire
  Récupérer en mémoire globale le nombre aléatoire à tester.
  Considérer que le nombre est premier.
  Pour chaque nombre premier de la liste en mémoire partagée faire
    Si le nombre aléatoire est multiple de ce nombre alors
      Le nombre n'est pas premier.
  Si le nombre n'est pas premier alors
    Le marquer comme tel en mémoire globale.
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Syntaxe de l'utilisation de la mémoire partagée

- Définir de la mémoire partagée : `__shared__`.
- Définir de la mémoire partagée de taille fixe.
  - Spécifier uniquement dans le kernel, taille statique :

```
__shared__ int tabShared[ 50 ];
```

- Définir de la mémoire partagée de taille variable (ex : dépend du nombre de nombres premiers calculés).
  - Tableau déclaré dans le kernel sans sa taille :

```
extern __shared__ int tabShared[];
```

  - Spécification de la taille de mémoire partagée utilisée en troisième « pseudo-paramètre » de l'appel du kernel :

```
monKernel<<< dimGrille, dimBloc, tailleShrd >>>();
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Code du kernel vérifiant la primalité d'un nombre (version mémoire partagée)

```
__global__ void filtreNbPremiers(
                    int * inoutListeNombresAleatoires, const int inNbNombresAleatoires,
              const int * inListeNombresPremiers,      const int inNbNombresPremiers )
{
   __shared__ int listeNbresPremiersShrd[ inNbNombresPremiers ];
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( threadIdx.x < inNbNombresPremiers )
      listeNbresPremiersShrd[ threadIdx.x ] = inListeNombresPremiers[ threadIdx.x ];

   __syncthreads();
   if ( indiceGlobal < inNbNombresAleatoires )
   {
      float   nombreAleatoire    = inoutListeNombresAleatoires[ indiceGlobal ];
      bool    estNombrePremier   = true;

      int     i                  = 0;
      while ( estNombrePremier && i < inNbNombresPremiers )
      {
         estNombrePremier   = nombreAleatoire % listeNbresPremiersShrd[ i++ ] == 0;
      }
      if ( !estNombrePremier )
         inoutListeNombresAleatoires[ indiceGlobal ] = -1;
   }
}
```

/!\ Code incorrect
mémoire dynamique mal allouée

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Kernel modifié et appel

```
__global__ void filtreNbPremiers(
                    int * inoutListeNombresAleatoires, const int inNbNombresAleatoires,
              const int * inListeNombresPremiers,      const int inNbNombresPremiers )
{
   extern __shared__ int listeNbresPremiersShrd[];
   [...]
}

std::vector< int > genererNombresPremiersAleatoires( const int inNbTirages,
                                                     const int inMax )
{
   [...]
   // Appel du kernel
   dim3   dimBloc( inNbTirages > 256 ? 256 : inNbTirages );
   dim3   dimGrille( inNbTirages / 256 + ( inNbTirages % 256 != 0 ) );
   int    tailleShared = nbNombrePremiers * sizeof( int );

   filtreNombresPremiers<<< dimGrille, dimBloc, tailleShared >>>(
                                                              nombresAleatoires_GPU,
                                                              inNbTirages,
                                                              nombresPremiers_GPU,
                                                              nbNombrePremiers
                                                             );
   [...]
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

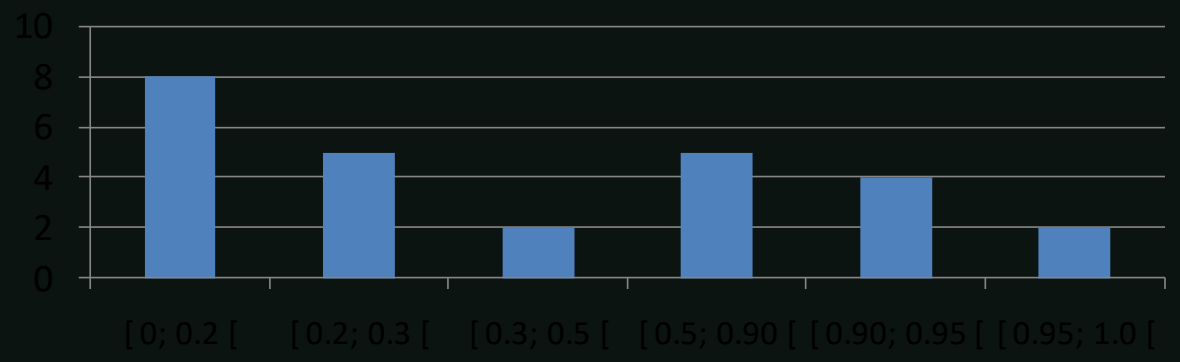
TP : histogramme de répartition

- Créer l'histogramme de répartition d'un tableau de nombre compris entre 0 (compris) et 1 (exclu).

Tableau de valeurs :	.42	.13	.28	.92	.64	.3	...	.01

	0	1	2	3	4
Intervalles de répartition :	.2	.3	.5	.9	.95

Tableau mis à jour :	2	0	1	4	3	2	...	0



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exercice : histogramme de répartition
Kernel mémoire globale uniquement

```
__global__ void creationHisto( float * inoutTabVal,    const int inNbValeurs,
                               const int *   inTabIntervals, const int inNbIntervals )
{
   int indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( indiceGlobal < inNbValeurs )
   {
      float valeur = inoutTabVal[ indiceGlobal ];

      int i = 0;
      while ( i < inNbIntervals && valeur >= inTabIntervals[ i ] )
      {
         ++i;
      }

      inoutTabVal[ indiceGlobal ] = i;
   }
}

creationHisto<<< nbBloc, nbThread >>>( tabVal_GPU,       NbValeurs,
                                       tabIntervals_GPU, NbIntervals );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exercice : histogramme de répartition
Kernel mémoire partagée

```
__global__ void creationHisto_Shared(
                                float * inoutTabVal,    const int inNbValeurs,
                          const int *   inTabIntervals, const int inNbIntervals )
{

}
```

creationHisto_Shared

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exercice : histogramme de répartition
Kernel mémoire partagée - Correction 1/2

```
__global__ void creationHisto_Shared( float * inoutTabVal,    const int inNbValeurs,
                                      float * inTabIntervals, const int inNbIntervals )
{
   extern __shared__ float tabInterValsShared[];
   int   indiceGlobal = threadIdx.x + blockIdx.x * blockDim.x;

   if ( threadIdx.x < inNbIntervals )
   {
      tabInterValsShared[ threadIdx.x ] = inTabIntervals[ threadIdx.x ];
   }
   __syncthreads();

   if ( indiceGlobal < inNbValeurs )
   {
      float valeur = inoutTabVal[ indiceGlobal ];

      int i = 0;
      while ( i < inNbIntervals && valeur >= tabInterValsShared[ i ] )
      {
         ++i;
      }

      inoutTabVal[ indiceGlobal ] = i;
   }
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exercice : histogramme de répartition
Kernel mémoire partagée - Correction 2/2

- Appel du kernel :

```
creationHisto_Shared<<< nbBloc, nbThread, NbIntervals * sizeof( float ) >>>(
                                                tabVal_GPU,        NbValeurs,
                                                tabIntervals_GPU, NbIntervals
                                                                  );
```

- Peu de travail pour des gains potentiellement importants
  - Dépend toujours des compute capabilities
- Mémoire présente en petite quantité !
  - Utiliser les autres mémoires disponibles

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Autres types de mémoire

Les différents types de mémoire restants

- Les registres : utilisés pour stocker les variables automatiques.
- La mémoire locale : utilisée pour les variables locales à un thread lorsqu'elles sont trop nombreuses ou de tailles trop importantes.
- La mémoire constante et la mémoire des textures : utilisées pour les accès en lecture seule à travers un cache.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Les registres

- Accès très rapide (aucune latence).
- Gérer automatiquement par le compilateur.
  - Déclaration de variables automatiques.
  - Ex : « int i; »
- En beaucoup trop petit nombre.
  - Compute capacity >= 5.0 : 64K registres 32 bits par SM.
  - Jusqu'à 1024 threads ordonnancés en même temps sur un SM.
  - => potentiellement 64 registres (variables automatiques) par threads !

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Mémoire locale

- Notion logicielle : surcouche de la mémoire globale.
- Utilisée lorsque les registres ne suffisent pas.
  - Ex : Tableaux trop grand ou pour lesquels le compilo’ n’est pas sûr de la taille
- Caractéristiques de la mémoire globale :
  - Présente en grande quantité.
  - Accessible en lecture et en écriture.
  - Extrêmement lente (de l’ordre de plusieurs centaines de cycle).

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

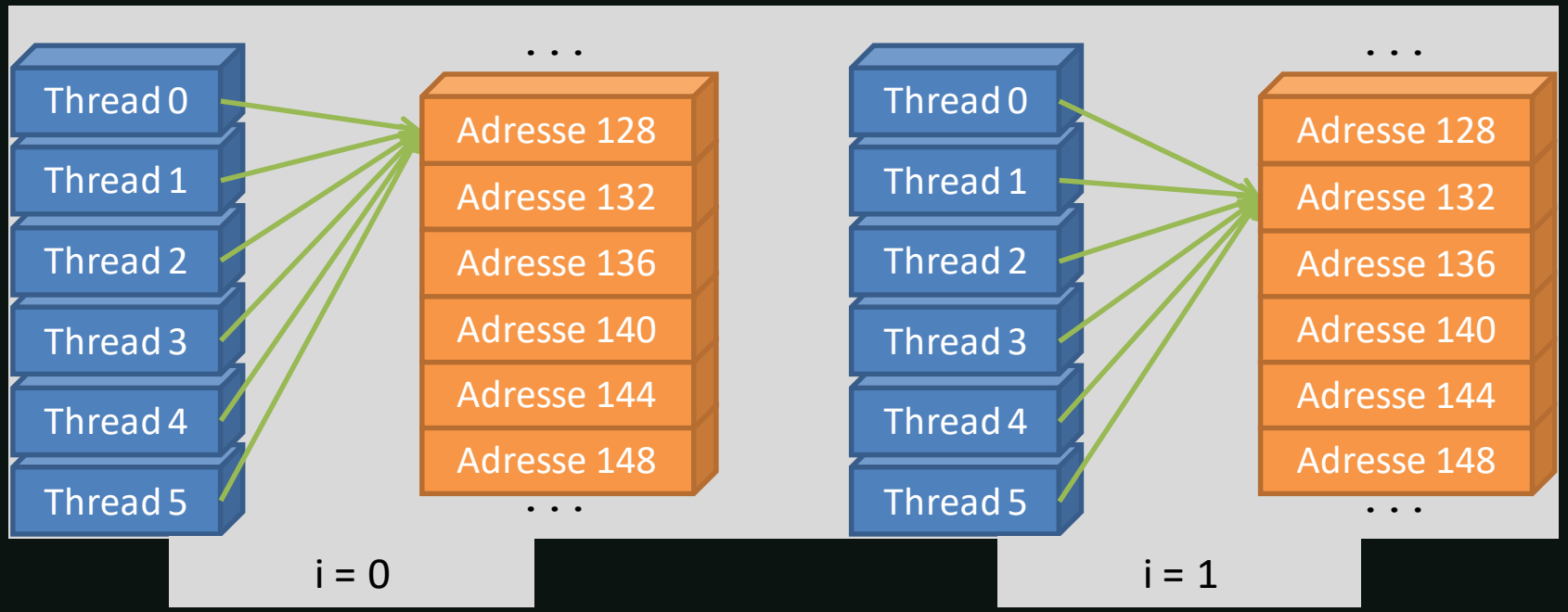
Mémoire constante

- Accès en lecture uniquement.
- Utilisation d’un cache au dessus de la mémoire globale.
  - Taille de la mémoire utilisée limitée (64 Ko).
  - Taille du cache sur chaque SM limitée (8 Ko).
  - Aucune évolution depuis 15 ans.
- Principe de localité temporelle : réutilisation des données déjà lues.
  - Tous les threads accèdent aux même données.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d'accès en localité temporelle

```
int i = 0;
while ( estNombrePremier && i < inNbNombresPremiers )
{
   estNombrePremier =
         nombreAleatoire % inListeNombresPremiers[ i ] == 0;
   i++;
}
```



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Implémentation

- Mot clé : `__constant__`.
- Déclaration du tableau en variable globale (accessible par le kernel et l'appelant).
- Copie (depuis le CPU vers le GPU uniquement car en lecture seule) à l'aide de la fonction `cudaMemcpyToSymbol`.
- Accès classique depuis le GPU.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d'utilisation de la mémoire constante

```
__constant__ int nombresPremiers_constant[ 100 ];
__global__ void filtreNbPremiers(
                    int * inoutListeNombresAleatoires, const int inNbNombresAleatoires,
            /*const int * inListeNombresPremiers,*/    const int inNbNombresPremiers )
{
   [...]
         estNombrePremier   = nombreAleatoire % nombresPremiers_constant[ i ] == 0;
}

   {
   [...]
   if ( nbNombrePremiers >= 100 )
      exit( 1 );
   cudaMemcpyToSymbol( nombresPremiers_constant, nombresPremiers_CPU,
                       nbNombrePremiers * sizeof( float ), 0,
                       cudaMemcpyHostToDevice );
   [...]
   // Appel du kernel
   filtreNombresPremiers<<< dimGrille, dimBloc >>>(nombresAleatoires_GPU, inNbTirages, nbNombrePremiers);
   [...]
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

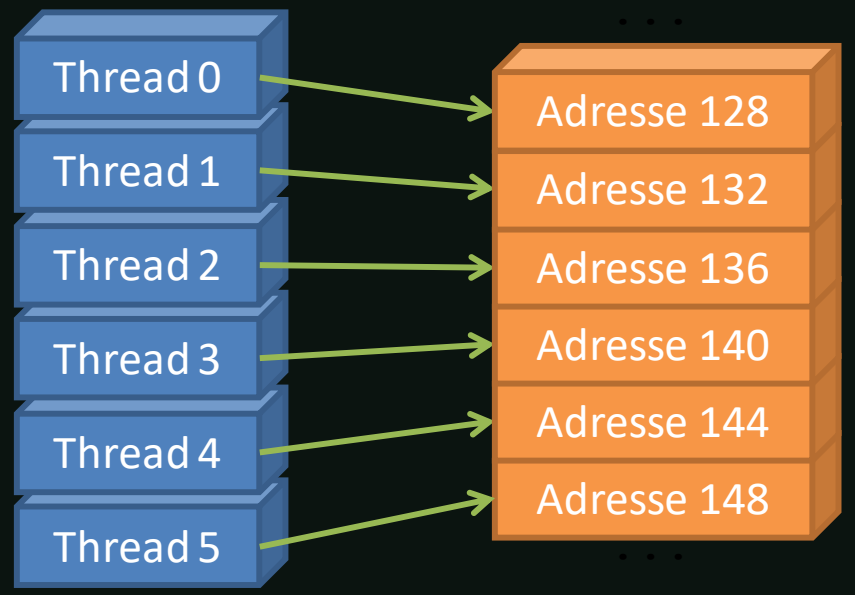
Mémoire des textures

- Accès en lecture uniquement.
- Utilisation d’un cache au dessus de la mémoire globale.
  - Taille plus importante que la mémoire constante.
  - Performances moins bonnes également.
- Principe de localité spatiale : mise en cache des données proches de celles déjà lues.
  - Les threads accèdent à des données se suivant (selon une, deux ou trois dimensions).
  - Optimisé pour la 2D.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d'accès en localité spatiale

```
float   nombreAleatoire  =
        inoutListeNombresAleatoires[ indiceGlobal ];
```



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Implémentation

- Beaucoup plus compliqué que la mémoire constante.
  - Cache configurable.

- Étapes nécessaires :

1. Créer le tableau normalement en mémoire globale.
2. Configurer un cache de texture.
3. Attacher le cache au tableau.
4. Passer le cache en paramètre du kernel.
5. Accéder à la mémoire à travers le cache.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Quel type de mémoire utilisé ? 1/2

Type de mémoire	Visibilité	Temps d'accès
Registre	Thread	1 cycle
Mémoire locale	Thread	> 100 cycles
Mémoire partagée	Bloc	1 cycle
Mémoire constante	Globale, cachée	1 cycle à > 100 cycles
Mémoire texture	Globale, cachée	1 cycle à > 100 cycles
Mémoire globale	Globale	> 100 cycles

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Quel type de mémoire utilisé ? 2/2

- Variables automatiques : registres (attention, s'il y en a trop : mémoire local !).
- Lecture seule en 1D : mémoire constante.
- Lecture seule en 2D ou 3D : mémoire des textures.
- Mémoire partageable entre les threads d'un bloc : mémoire partagée.
- Lecture/Écriture : mémoire globale.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Gestion des erreurs

Introduction sur les erreurs

- Code exécuté sur le GPU.
- Débogage :
  - Oublier les « std::cerr << "toto« << std::endl; » dans les kernels.
  - Utilisation du cuda-gdb sous Linux, Nsight sous Windows.
- Erreur lors des appels de fonction :
  - Pas d’exception (ex : std::bad_alloc).
  - Aucune garantie qu’une fonction compilée possède les bons paramètres dynamiques.
  - => A l’ancienne : gestion du code d’erreur.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Erreurs dynamiques

- A chaque appel sur le GPU : récupération possible d’un code d’erreur.
- Traiter cette erreur :
  - Lors du développement pour ne pas passer son temps à corriger un kernel qui n’est jamais lancé.
  - Lors de l’exécution de l’application finale, informer l’utilisateur d’un problème/incompatibilité avec sa carte GPU.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Appels sur le GPU

- Copie mémoire CPU vers GPU, GPU vers CPU, GPU vers GPU…
- Allocation mémoire sur le GPU.
- Appel de kernel.
- Appel de fonction donnant accès à des informations sur la carte GPU (ex : getDevices…).

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Appel synchrone / appel asynchrone

- Appel synchrone
  - Appel bloquant.
  - Attente du résultat de la fonction.
    - Ex : Dans un code classique, appel d’une fonction quelconque.

- Appel asynchrone
  - Appel non bloquant.
  - Possibilité d’exécuter un autre code en attendant la réponse.
  - Informer de la fin de la fonction par message.
    - Ex : Requête GET en HTML.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Récupération de l’erreur

- Deux cas différents : appels synchrones et appels asynchrones sur la carte GPU.
- Appels synchrones :
  - Allocation mémoire.
  - Copie de mémoire classique entre CPU et GPU.
  - …
  - => Récupération du code erreur en sortie de la fonction.
- Appels asynchonres :
  - Appel de kernels.
  - Copie de mémoire asynchrone entre CPU et GPU.
  - …
  - => Récupération du code erreur avec getCudaLastError.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple appel synchrone

- Exemple de l’allocation de mémoire :

```
int tailleTab = 100000000000;
float * tabCPU = new float[ tailleTab ]; // Traiter l’exception
float * tab_GPU;
cudaError_t erreur = cudaMalloc(
                       &tab_GPU,
                       tailleTab * sizeof( float )
                     );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple appel asynchrone

- Exemple de l'appel de kernel :

```
dim3          dimGrille( 10 );
dim3          dimBloc( 10000 );
mon_kernel<<< dimGrille, dimBloc >>>();
cudaError_t erreur = getCudaLastError();
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Le type cudaError_t 1/2

typedef enum cudaError cudaError_t;

- Documentation officielle de cudaError :

enum cudaError

CUDA error types

Enumerator:

cudaSuccess No errors.

cudaErrorMissingConfiguration Missing configuration error.

cudaErrorMemoryAllocation Memory allocation error. [...]

cudaErrorAddressOfConstant Address of constant error Deprecated: This error return is deprecated as of Cuda 3.1. Variables in constant memory may now have their address taken by the runtime via cudaGetSymbolAddress().

[...]

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Le type cudaError_t 2/2

- Un type pour toutes les erreurs (synchrones et asynchrones).

  Beaucoup de codes différents dans un seul enum.

- Inutile de connaitre par cœur toutes les valeurs.

- Une seule à connaitre obligatoirement : le code correspondant à l’absence d’erreur : cudaSuccess.

- Plusieurs erreurs intéressantes (qui reviennent souvent) : cudaErrorLaunchFailure, cudaErrorNoDevice…

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Traitement minimaliste des erreurs

- Tester l'égalité à la valeur cudaSuccess.
- Arrêter l'application en cas de valeurs différentes.

```
cudaError_t erreur = cudaMalloc(
                        &tab_GPU,
                        tailleTab * sizeof( float )
                      );
if ( erreur != cudaSuccess )
{
   std::cerr << "Erreur a la copie" << std::endl;
   exit( 1 );
}
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Traitement plus complet

- Afficher le message correspondant à l’erreur.
  - Fonction CUDA maintenue à jour : const char * cudaGetErrorString(cudaError_t);
- Traiter réellement les erreurs selon leurs valeurs.
  - Ex :
    - cudaErrorDevicesUnavailable : Recommencer l’opération après un temps d’attente.
    - cudaErrorNoDevice : Passer à l’algorithme séquentiel.
    - …

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement

Qu’est ce que l’ordonnancement

- Problématique :
  - Plusieurs tâches à effectuer dans un ordre quelconque.
- Ordonnancement : fonction d’ordonner l’affectation des différentes tâches aux différentes ressources.
  - Tâches GPU : warps.
  - Ressources GPU : Streaming Multiprocessor.
- But : réaliser l’ensemble des tâches le plus rapidement possible.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Utilité de l’ordonnancement

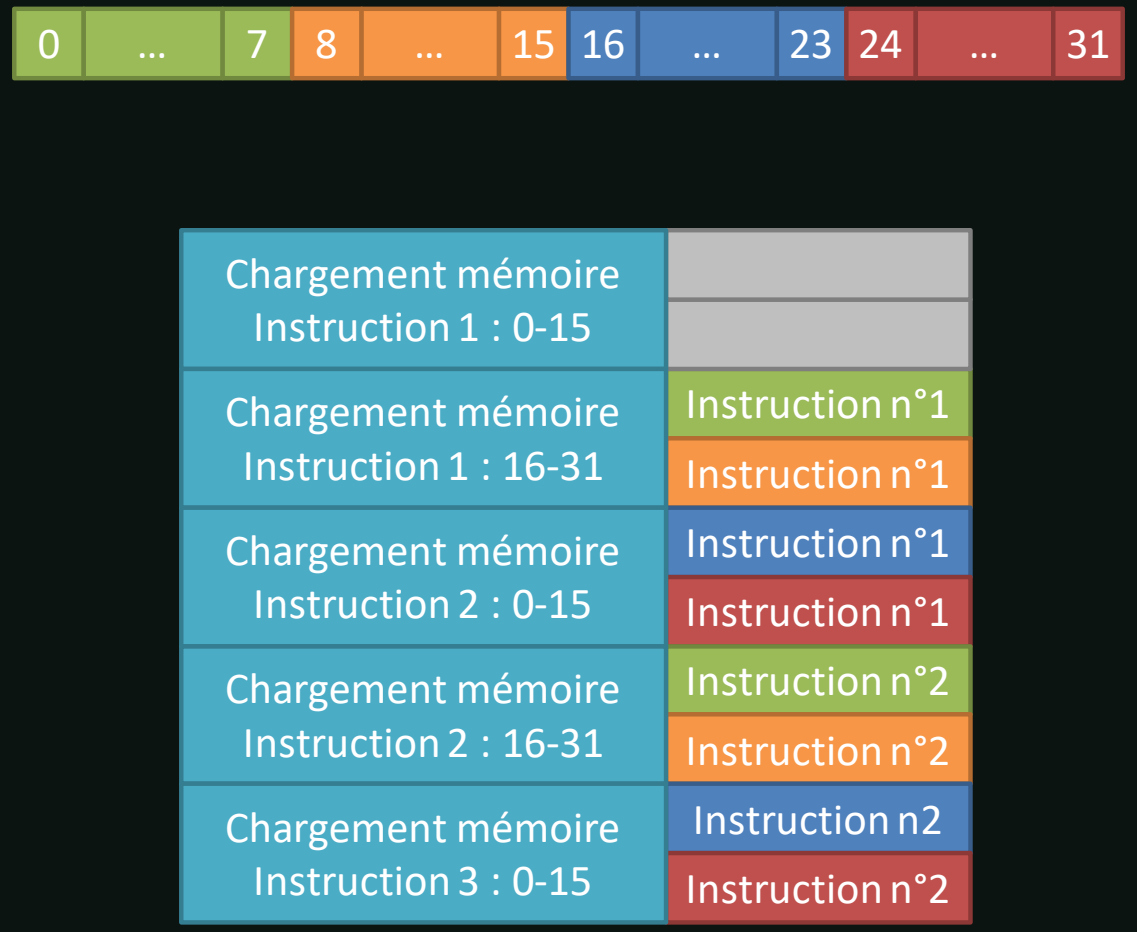
- Important pour masquer les accès mémoires ou les demandes de calcul complexe.
  - => Pourvoir exécuter un warp différent pendant qu’un warp attend sa ressource.

- Plus le nombre de warps ordonnancés est grand, plus les chances sont grandes de trouver un warp qui peut être exécuté pendant qu’un autre attend.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple schématique

- 1 warp, 32 threads
- Instruction complexe
- 16 SFU (Special Function Unit)



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement des warps 1/3

- Environ 1024 threads maximum ordonnancés en même temps sur un SM (32 warps) (dépend des CC).
- 32 SM sur l'ensemble d'une carte (dépend de la carte) : jusqu'à 32728 threads ordonnancés en même temps.
- Objectif : essayer de s'approcher le plus possible de ce nombre de threads ordonnancé en même temps sur la carte GPU.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement des warps 2/3

- Warps d'un bloc ordonnancé dans n'importe quel ordre (tant qu'aucun ne requiert d'accès ayant une latence ou de synchronisation).

1 bloc de 128 threads : 4 warps

Warp 1	Warp 2
Warp 3	Warp 4

Kernel :
Instruction n°1
Instruction n°2
Instruction n°3
Instruction n°4

Warp 1 – Instruction 1
Warp 1 – Instruction 2
Warp 2 – Instruction 1
Warp 4 – Instruction 1
Warp 1 – Instruction 3
Warp 3 – Instruction 1
Warp 1 – Instruction 4
Warp 4 – Instruction 2

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement des warps 3/3

- Warps d'un bloc ordonnancé dans n'importe quel ordre (tant qu'aucun ne requiert d'accès ayant une latence ou de synchronisation).

1 bloc de 128 threads : 4 warps

Warp 1	Warp 2
Warp 3	Warp 4

Kernel :
Instruction n°1
Instruction n°2
__syncthread();
Instruction n°3
Instruction n°4

Warp 1 – Instruction 1
Warp 1 – Instruction 2
Warp 2 – Instruction 1
Warp 4 – Instruction 1
Warp 3 – Instruction 1
Warp 4 – Instruction 2
Warp 3 – Instruction 2
Warp 2 – Instruction 2
Syncthread();
Warp 1 – Instruction 3

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement des blocs 1/2

- Plusieurs blocs ordonnancés en même temps sur un même SM.
- Toujours pas plus de 1024 threads malgré cela.
- Possibilité d’atteindre l’occupation maximal sans avoir des blocs trop grand.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Ordonnancement des blocs 2/2

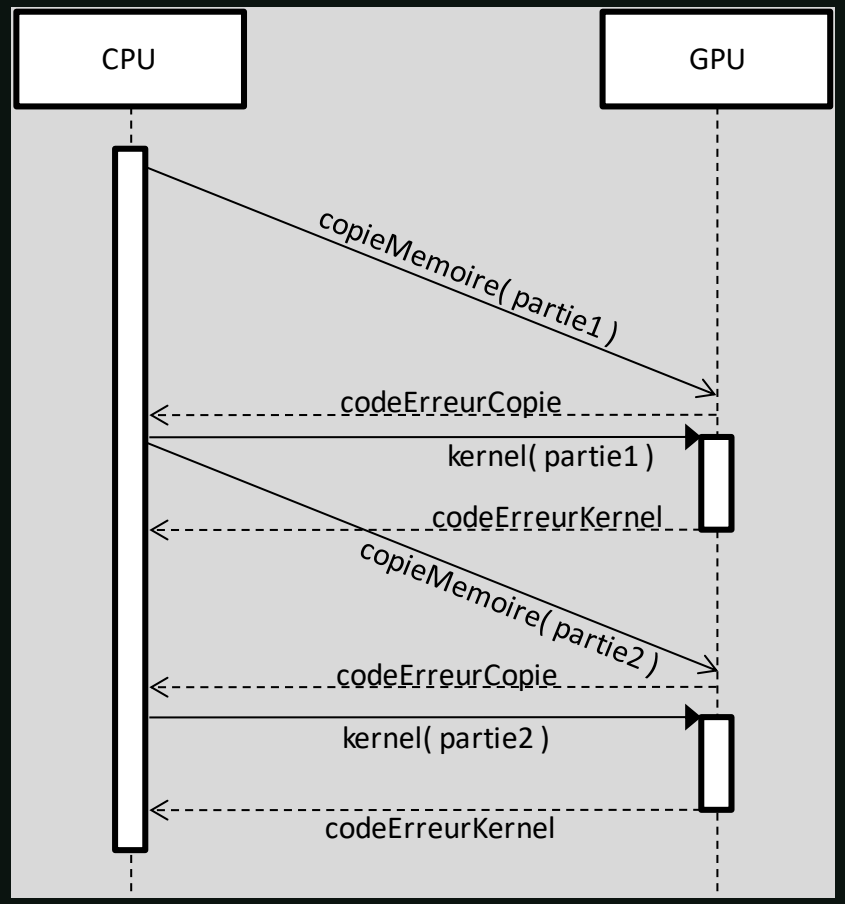
- Limitation à travers d'autres limites des SM.
  - Mémoire partagée
  - Registres
  - …
- Bien identifier l'utilisation des composantes en nombres limitées.
  - Utilisation du profiler de NVIDIA.

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Flux asyncrhone

Utilisation des flux asynchones

- Objectif : ne pas perdre de temps à attendre la copie de la mémoire.
- Méthode :
  - Lancer la copie de façon asynchrone.
  - Lors de l'appel d'un kernel :
    - Si la mémoire est copiée : lancer le kernel.
    - Sinon attendre que ce soit fait pour lancer le kernel.



© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Implémentation en CUDA

- Algorithme de l’utilisation de copie asynchrone en CUDA :
  - Instanciation d’un flux : cudaStreamCreate.
  - Copie de la mémoire en rajoutant le flux en paramètre : cudaMemcpyAsync(dest, src, taille, sens, flux).
  - Appel du kernel en le paramétrant avec le flux : monKernel<<< dimGrille, dimBloc, tailleShared, flux>>>().

© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d’utilisation de 5 flux

```
int           nbFlux   = 5;
cudaStream_t  tabStream[ nbFlux ];
for ( int i = 0; i < nbFlux; ++i )
  cudaStreamCreate( &tabStream[ i ] );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d’utilisation de 5 flux

```
int tailleParFlux = taille_tab_host / nbFlux;
float * tab_device;
cudaMalloc( (void **) & tab_device, taille_tab_host * sizeof( float ) );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d’utilisation de 5 flux

```
for ( int i = 0; i < nbFlux; ++i )
    cudaMemcpyAsync(
                    tab_device + i * tailleParFlux * sizeof( float ),
                    tab_host + i * tailleParFlux * sizeof( float ),
                    tailleParFlux * sizeof( float ),
                    cudaMemcpyHostToDevice,
                    tabStream[ i ]
                  );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d’utilisation de 5 flux

```
for ( int i = 0; i < nbFlux; ++i )
    monKernel<<< 1, tailleParFlux, 0, tabStream[ i ] >>>
        ( tab_device + i * tailleParFlux * sizeof( float ), tailleParFlux );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.

Exemple d’utilisation de 5 flux

```
int           nbFlux   = 5;
cudaStream_t  tabStream[ nbFlux ];
for ( int i = 0; i < nbFlux; ++i )
  cudaStreamCreate( &tabStream[ i ] );

int tailleParFlux = taille_tab_host / nbFlux;
float * tab_device;
cudaMalloc( (void **) & tab_device, taille_tab_host * sizeof( float ) );
for ( int i = 0; i < nbFlux; ++i )
    cudaMemcpyAsync(
                      tab_device + i * tailleParFlux * sizeof( float ),
                      tab_host + i * tailleParFlux * sizeof( float ),
                      tailleParFlux * sizeof( float ),
                      cudaMemcpyHostToDevice,
                      tabStream[ i ]
                    );

for ( int i = 0; i < nbFlux; ++i )
    monKernel<<< 1, tailleParFlux, 0, tabStream[ i ] >>>( tab_device + i * tailleParFlux * sizeof( float ), tailleParFlux );
```


© 2023 Adobe. All Rights Reserved. Adobe Confidential.